

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA KỸ THUẬT ĐIỆN TỬ 2

BÀI GIẢNG MÔN HỌC

KIẾN TRÚC MÁY TÍNH

CHƯƠNG 1 GIỚI THIỆU CHUNG

*Tài liệu lưu hành nội bộ dùng cho sinh viên hệ đại học chính quy
ngành Công nghệ Kỹ thuật Điện tử – Viễn thông*

MỤC LỤC

1	Giới thiệu chung	1
1.1	Máy tính và phân loại máy tính	1
1.1.1	Khái niệm máy tính	2
1.1.2	Phân loại máy tính	2
1.1.3	Mô hình cơ bản của máy tính	4
1.2	Kiến trúc và tổ chức máy tính	6
1.2.1	Kiến trúc tập lệnh	6
1.2.2	Tổ chức máy tính	7
1.2.3	Phần cứng	7
1.3	Hoạt động cơ bản của máy tính	8
1.3.1	Thực hiện chương trình	8
1.3.2	Hoạt động ngắt	10
1.3.3	Hoạt động vào-ra	13
1.4	Lịch sử phát triển máy tính	14
1.4.1	Thế hệ thứ nhất: đèn điện tử chân không	14
1.4.2	Thế hệ thứ hai: tranzito	15
1.4.3	Thế hệ thứ ba: vi mạch tích hợp	15
1.4.4	Thế hệ thứ tư: vi mạch cỡ lớn và bộ vi xử lý	15
1.5	Một số kiến trúc máy tính cơ bản	16
1.5.1	Kiến trúc von Neumann	16
1.5.2	Kiến trúc Harvard	19
1.6	Các hệ đếm và tổ chức lưu trữ dữ liệu trên máy tính	20
1.6.1	Nguyên tắc chung của hệ đếm theo vị trí	20
1.6.2	Hệ thập phân	20
1.6.3	Hệ nhị phân	21
1.6.4	Hệ thập lục phân	23
1.6.5	Đơn vị dữ liệu và thông tin trong máy tính	24
1.6.6	Biểu diễn số nguyên	25

1.6.7	Biểu diễn số thực	29
1.6.8	Biểu diễn ký tự	34
	Phụ lục: Lịch sử phát triển của các bộ vi xử lý	37
	Câu hỏi ôn tập	41
	Bài tập	42
	Đáp án và hướng dẫn giải	44

CHƯƠNG 1

GIỚI THIỆU CHUNG

MỤC TIÊU CỦA CHƯƠNG

Sau khi học xong chương này, sinh viên có khả năng:

- 1) Trình bày được khái niệm máy tính, bốn chức năng cơ bản và các tiêu chí phân loại máy tính hiện nay.
- 2) Phân biệt được ba mức nghiên cứu một hệ thống máy tính: kiến trúc tập lệnh, tổ chức máy tính và phần cứng.
- 3) Mô tả được chu trình lệnh, cơ chế ngắt và các phương pháp trao đổi dữ liệu vào-ra.
- 4) Nêu được các mốc chính trong lịch sử phát triển máy tính và bộ vi xử lý.
- 5) So sánh được kiến trúc von Neumann với kiến trúc Harvard.
- 6) Thực hiện thành thạo việc chuyển đổi giữa các hệ đếm; biểu diễn được số nguyên có dấu theo mã bù hai, số thực theo chuẩn IEEE 754 và ký tự theo các bộ mã thông dụng.

Chương mở đầu này cung cấp bức tranh tổng quát về đối tượng nghiên cứu của môn học. Nội dung được chia thành sáu phần. Ba phần đầu xây dựng bộ khái niệm nền tảng: máy tính là gì, nghiên cứu máy tính ở những mức nào và máy tính vận hành ra sao. Phần thứ tư điểm lại quá trình phát triển của ngành, qua đó cho thấy các quyết định thiết kế hôm nay đều có gốc rễ lịch sử của nó. Phần thứ năm giới thiệu hai mô hình kiến trúc kinh điển vẫn đang chi phối mọi máy tính hiện đại. Phần cuối cùng, dài nhất và mang tính công cụ, trình bày cách biểu diễn thông tin bên trong máy tính; đây là phần sinh viên phải nắm thật chắc vì toàn bộ các chương sau đều sử dụng đến.

Người học nên đọc chương này cùng với việc tự làm lại các ví dụ tính toán. Riêng phần biểu diễn dữ liệu, kinh nghiệm cho thấy chỉ đọc hiểu là chưa đủ, phải luyện đến mức chuyển đổi cơ số và lấy bù hai một cách tự động thì mới theo kịp các chương về tập lệnh và đơn vị số học sau này.

1.1 Máy tính và phân loại máy tính

1.1.1 Khái niệm máy tính

Định nghĩa 1.1: Máy tính

Máy tính (*computer*) là thiết bị điện tử thực hiện tự động một dãy các lệnh đã được lưu sẵn bên trong nó, nhằm mục đích tiếp nhận, xử lý, lưu trữ và đưa ra dữ liệu.

Định nghĩa ngắn gọn ở trên đã hàm chứa bốn chức năng cơ bản mà bất kỳ máy tính nào, từ vi điều khiển 8 bit trong chiếc lò vi sóng cho tới siêu máy tính nhiều nghìn nút, đều phải có:

- (1) **Nhận dữ liệu vào.** Dữ liệu đến từ bàn phím, cảm biến, ổ đĩa, đường truyền mạng hoặc từ một máy tính khác.
- (2) **Xử lý và lưu trữ dữ liệu** theo dãy lệnh đã được ghi sẵn trong bộ nhớ. Đây là chức năng làm nên bản chất của máy tính: nó không thực hiện một phép biến đổi cố định do phần cứng quy định, mà thực hiện đúng những gì dãy lệnh yêu cầu.
- (3) **Đưa dữ liệu ra** tới màn hình, máy in, cơ cấu chấp hành hoặc các thiết bị ngoại vi khác.
- (4) **Điều khiển** toàn bộ ba hoạt động trên theo đúng trình tự và đúng thời điểm.

Tập hợp các lệnh nằm trong bộ nhớ, được sắp xếp theo một trình tự xác định để yêu cầu máy tính giải quyết một công việc cụ thể, gọi là **chương trình** (*program*). Chính khả năng thay chương trình mà không phải thay phần cứng đã làm cho máy tính trở thành thiết bị vạn năng. Trước khi bộ vi xử lý ra đời, mỗi mạch số được thiết kế cho một chức năng cố định; muốn đổi chức năng thì phải thiết kế lại mạch. Ý tưởng đưa chức năng vào phần mềm thay vì đúc cứng vào phần cứng là bước ngoặt lớn nhất của ngành, và ta sẽ trở lại ý tưởng này ở mục 1.5.1.

Cần phân biệt *dữ liệu* với *thông tin*. Dữ liệu là các dãy bit thô mà máy tính lưu và xử lý; thông tin là ý nghĩa mà con người gán cho các dãy bit đó theo một quy ước biểu diễn nào đó. Cùng một byte 0100 0001, nếu hiểu theo mã ASCII thì đó là ký tự A, hiểu theo số nguyên không dấu thì là 65, còn nếu bộ xử lý nhận nó trong pha nhận lệnh thì đó lại là một mã lệnh. Bản thân máy tính không tự biết dãy bit đó mang ý nghĩa gì; ý nghĩa nằm ở ngữ cảnh sử dụng do chương trình quy định.

1.1.2 Phân loại máy tính

Có nhiều cách phân loại. Cách phổ biến hiện nay dựa trên mục đích sử dụng và quy mô hệ thống, chia máy tính thành ba nhóm lớn.

Máy tính cá nhân (Personal Computers)

Gồm máy để bàn (*desktop*) và máy xách tay (*laptop*), phục vụ trực tiếp một người dùng. Đặc trưng của nhóm này là tính đa dụng: cùng một máy có thể soạn thảo văn bản, lập trình, xem phim

hay chơi trò chơi. Tiêu chí thiết kế quan trọng nhất là tỷ số hiệu năng trên giá thành. Máy tính bảng và điện thoại thông minh, xét về kiến trúc, cũng thuộc nhóm này dù hình dáng bên ngoài đã khác đi nhiều.

Máy chủ (Servers)

Là máy phục vụ, đặt trong mạng để quản lý tài nguyên và cung cấp dịch vụ cho nhiều máy khách cùng lúc. Máy chủ ít khi có màn hình hay bàn phím riêng, người quản trị truy nhập từ xa. Hai yêu cầu hàng đầu là hiệu năng và độ tin cậy: một máy chủ hỏng có thể làm ngừng dịch vụ của hàng nghìn người dùng, nên hệ thống thường có nguồn kép, ổ đĩa RAID, bộ nhớ ECC tự sửa lỗi. Giá một máy chủ trải rộng từ vài nghìn tới hàng triệu đô-la Mỹ tùy quy mô. Ở mức cao nhất của nhóm này là siêu máy tính và trung tâm dữ liệu.

Máy tính nhúng (Embedded Computers)

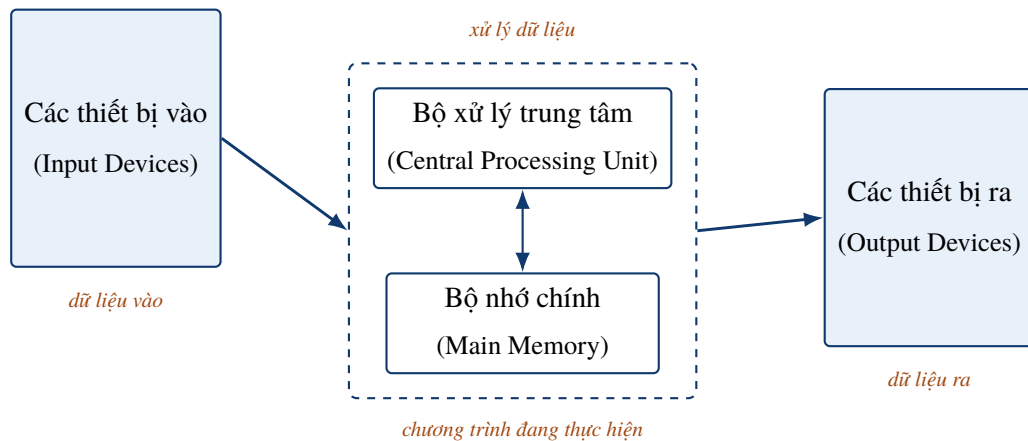
Được đặt ẩn bên trong một thiết bị khác và điều khiển thiết bị đó. Người sử dụng thường không nhận ra mình đang dùng máy tính khi bấm nút máy giặt hay khởi động xe máy. Máy tính nhúng được thiết kế chuyên dụng cho một nhiệm vụ duy nhất, nên các tiêu chí thiết kế cũng khác hẳn: công suất tiêu thụ thấp, giá thành cực rẻ, kích thước nhỏ, đáp ứng thời gian thực. Xét về số lượng, đây là nhóm đông đảo nhất; mỗi năm có hàng chục tỷ bộ vi điều khiển được xuất xưởng, gấp nhiều lần tổng số máy tính cá nhân và máy chủ cộng lại.

Bảng 1.1. So sánh khái quát ba nhóm máy tính

Tiêu chí	Máy tính cá nhân	Máy chủ	Máy tính nhúng
Mục đích	Đa dụng	Cung cấp dịch vụ	Chuyên dụng, một nhiệm vụ
Số người dùng	Một	Rất nhiều	Không có người dùng trực tiếp
Khoảng giá	Vài trăm đến vài nghìn USD	Vài nghìn đến hàng triệu USD	Từ dưới một USD đến vài trăm USD
Tiêu chí thiết kế chính	Hiệu năng trên giá thành	Thông lượng, độ tin cậy, khả năng mở rộng	Công suất, giá, thời gian đáp ứng
Sản lượng hàng năm	Hàng trăm triệu	Hàng triệu	Hàng chục tỷ

1.1.3 Mô hình cơ bản của máy tính

Dù thuộc nhóm nào, một máy tính đều có thể quy về mô hình ba khối chức năng kèm hệ thống liên kết như hình 1.1. Dữ liệu đi vào qua các thiết bị vào, được bộ xử lý trung tâm gia công dưới sự dẫn dắt của chương trình đang nằm trong bộ nhớ chính, rồi kết quả được đưa ra qua các thiết bị ra.



Hình 1.1. Mô hình cơ bản của máy tính

Bộ xử lý trung tâm

Bộ xử lý trung tâm (CPU) điều khiển mọi hoạt động của máy và trực tiếp thực hiện việc xử lý dữ liệu. Đây là thành phần phức tạp nhất trong hệ thống, bên trong nó gồm:

- **Đơn vị số học và logic** (ALU – Arithmetic Logic Unit): thực hiện các phép cộng, trừ, nhân, chia, so sánh, AND, OR, NOT, dịch bit.
- **Đơn vị điều khiển** (CU – Control Unit): giải mã lệnh và phát ra các tín hiệu điều khiển để các bộ phận khác phối hợp thực hiện lệnh.
- **Tập thanh ghi** (Register File): các ô nhớ tốc độ rất cao nằm ngay trong CPU, dùng chứa dữ liệu và các thông tin trạng thái đang được thao tác.
- **Bus bên trong CPU**: nối ba thành phần trên với nhau.

Bộ nhớ chính

Bộ nhớ chính lưu trữ cả lệnh lẫn dữ liệu của chương trình đang chạy. Nó được tổ chức thành một dãy các ngăn nhớ, mỗi ngăn có một địa chỉ riêng, thường mỗi ngăn chứa một byte. Về công nghệ, bộ nhớ chính gồm hai loại:

- **ROM** (Read Only Memory): chứa các lệnh và dữ liệu cố định của hệ thống, điển hình là chương trình khởi động. Thông tin trong ROM không mất khi ngắt điện.

- **RAM** (*Random Access Memory*): chứa lệnh và dữ liệu trong thời gian chương trình chạy. Thông tin trong RAM mất khi ngắt điện.

Tên gọi RAM dễ gây hiểu nhầm. "Truy nhập ngẫu nhiên" ở đây không có nghĩa là truy nhập lung tung, mà có nghĩa là thời gian truy nhập tới mọi ngăn nhớ đều như nhau, không phụ thuộc vị trí ngăn nhớ đó nằm ở đâu. Theo nghĩa này thì ROM cũng là bộ nhớ truy nhập ngẫu nhiên; điểm khác nhau thực sự giữa hai loại nằm ở khả năng ghi và ở tính bất biến khi mất nguồn.

Hệ thống vào-ra

Hệ thống vào-ra, còn gọi là các thiết bị ngoại vi (*peripheral devices*), làm nhiệm vụ trao đổi thông tin giữa máy tính với thế giới bên ngoài. Ta chia thành:

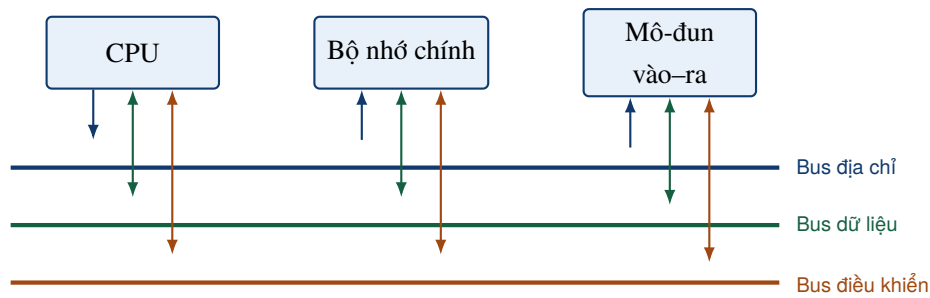
- **Thiết bị vào:** đưa dữ liệu và lệnh điều khiển vào hệ thống. Ví dụ bàn phím, chuột, máy quét, cảm biến.
- **Thiết bị ra:** đưa kết quả đã xử lý ra cho người dùng hoặc cho thiết bị khác. Ví dụ màn hình, máy in, loa.
- **Thiết bị vừa vào vừa ra:** ổ đĩa, card mạng, màn hình cảm ứng.

Mỗi thiết bị ngoại vi không nối thẳng vào bus hệ thống mà thông qua một **mô-đun vào-ra** đóng vai trò cầu nối. Mô-đun này chuyển đổi khuôn dạng tín hiệu, đệm dữ liệu và bù chênh lệch tốc độ giữa CPU rất nhanh với thiết bị ngoại vi rất chậm.

Bus hệ thống

Bus hệ thống (*system bus*) là tập hợp các đường dây cung cấp cơ chế truyền thông giữa ba thành phần trên. Theo chức năng, bus hệ thống chia thành ba loại:

- **Bus địa chỉ** (*address bus*): truyền tín hiệu địa chỉ từ CPU đến bộ nhớ và các mô-đun vào-ra để xác định ngăn nhớ hoặc cổng cần trao đổi. Bus này chỉ có một chiều đi ra từ CPU.
- **Bus dữ liệu** (*data bus*): truyền dữ liệu qua lại giữa CPU và các thành phần khác. Bus này hai chiều.
- **Bus điều khiển** (*control bus*): truyền các tín hiệu điều khiển từ CPU đi (đọc bộ nhớ, ghi bộ nhớ, đọc cổng vào-ra) và các tín hiệu trạng thái, tín hiệu yêu cầu ngắt từ các thành phần khác gửi về CPU.



Hình 1.2. Ba thành phần chức năng và ba loại bus của hệ thống

Số đường của bus địa chỉ quyết định dung lượng bộ nhớ tối đa mà bộ xử lý có thể quản lý, còn số đường của bus dữ liệu ảnh hưởng trực tiếp tới tốc độ trao đổi. Hai thông số này là căn cứ quan trọng khi đánh giá một bộ xử lý.

Ví dụ 1.1: Không gian địa chỉ và bus địa chỉ

Bộ vi xử lý 8086 có bus địa chỉ 20 đường và bus dữ liệu 16 đường. Hãy xác định dung lượng bộ nhớ tối đa mà nó quản lý được và lượng dữ liệu trao đổi trong một chu kỳ bus. So sánh với bộ xử lý có bus địa chỉ 32 đường.

Giải. Với n đường địa chỉ, CPU tạo ra được 2^n tổ hợp khác nhau, tương ứng 2^n ngăn nhớ được đánh địa chỉ. Do đó:

$$2^{20} = 1\,048\,576 \text{ ngăn nhớ} = 1 \text{ MB (với ngăn nhớ 1 byte).}$$

Bus dữ liệu 16 đường cho phép chuyển 2 byte trong một chu kỳ bus.

Với bus địa chỉ 32 đường: $2^{32} = 4\,294\,967\,296$ ngăn nhớ, tức 4 GB. Đây chính là lý do các hệ điều hành 32 bit bị giới hạn ở 4 GB bộ nhớ, một giới hạn đã từng là trở ngại thực tế trong những năm 2000 và là động lực chuyển sang kiến trúc 64 bit.

1.2 Kiến trúc và tổ chức máy tính

Khi nói "nghiên cứu một máy tính", người ta có thể đang nói tới những việc rất khác nhau. Người viết trình biên dịch quan tâm tới tập lệnh mà bộ xử lý hiểu được; người thiết kế chip quan tâm tới cách bố trí các tầng đường ống; người làm mạch in quan tâm tới mức điện áp và định thời tín hiệu. Để tránh lẫn lộn, ngành kiến trúc máy tính phân biệt ba mức nghiên cứu.

1.2.1 Kiến trúc tập lệnh

Kiến trúc tập lệnh (ISA – *Instruction Set Architecture*) nghiên cứu máy tính theo cách nhìn của người lập trình, cụ thể là người lập trình hợp ngữ hoặc người viết trình biên dịch. Nội dung của mức này gồm: tập lệnh và khuôn dạng lệnh, các kiểu dữ liệu được hỗ trợ, tập thanh ghi khả kiến với phần mềm, các chế độ địa chỉ, cách tổ chức bộ nhớ và cơ chế xử lý ngoại lệ.

Đặc điểm quan trọng nhất của ISA là tính ổn định. ISA đóng vai trò một bản giao ước giữa phần cứng và phần mềm: nhà sản xuất cam kết mọi bộ xử lý cùng họ đều hiểu đúng tập lệnh đó, nhờ vậy phần mềm viết cho máy cũ vẫn chạy được trên máy mới. Cùng một kiến trúc tập lệnh có thể có rất nhiều sản phẩm với tổ chức bên trong và công nghệ chế tạo hoàn toàn khác nhau.

1.2.2 Tổ chức máy tính

Tổ chức máy tính (*Computer Organization*), hay còn gọi là vi kiến trúc (*Microarchitecture*), nghiên cứu thiết kế máy tính ở mức cao: tổ chức bên trong CPU, phân cấp bộ nhớ và hệ thống cache, cấu trúc bus, cơ chế đường ống, cơ chế dự đoán rẽ nhánh. Đây là mức trả lời câu hỏi "làm thế nào để thực hiện được ISA đó cho nhanh và cho rẻ".

Người lập trình bình thường không nhìn thấy mức này. Chương trình của họ không cần biết bộ xử lý có bao nhiêu tầng đường ống hay cache L2 lớn bao nhiêu; chương trình vẫn chạy đúng. Nhưng chương trình chạy *nhANH hay chậm* thì lại phụ thuộc rất nhiều vào các quyết định ở mức tổ chức.

1.2.3 Phần cứng

Mức phần cứng (*Hardware*) nghiên cứu thiết kế logic chi tiết và công nghệ chế tạo, đóng gói: sơ đồ mạch logic, công nghệ bán dẫn, kích thước tranzito, số lớp kim loại, kiểu đế chip, cách tản nhiệt.

Bảng 1.2. Ba mức nghiên cứu một hệ thống máy tính

Mức	Câu hỏi trả lời	Ví dụ nội dung
Kiến trúc tập lệnh	Máy tính <i>làm được gì</i> theo con mắt người lập trình?	Tập lệnh x86-64, ARMv8, RISC-V; số thanh ghi; chế độ địa chỉ
Tổ chức máy tính	Thực hiện tập lệnh đó <i>bằng cách nào</i> ?	Số tầng đường ống, kích thước cache, đơn vị nhân dấu phẩy động
Phần cứng	Mạch điện <i>được chế tạo ra sao</i> ?	Công nghệ 7 nm, số lớp kim loại, kiểu đế LGA1700

Ví dụ 1.2: Một kiến trúc, nhiều tổ chức

Bộ vi xử lý Intel 8086 (1978) và một bộ xử lý Intel Core i thế hệ mới đều thuộc họ kiến trúc x86. Về mặt kiến trúc tập lệnh, các lệnh cơ bản như MOV, ADD, JMP của 8086 vẫn được bộ xử lý mới thực hiện đúng, nghĩa là một chương trình hợp ngữ viết cho 8086 về nguyên tắc vẫn chạy được.

Về mặt tổ chức thì hai bộ xử lý này khác nhau hoàn toàn. 8086 có khoảng 29 000 tranzito, không có cache, không có đường ống theo nghĩa hiện đại, thực hiện lệnh gần như tuần tự.

Bộ xử lý hiện đại có hàng tỷ tranzito, ba cấp cache, hàng chục tầng đường ống, thực hiện lệnh không theo thứ tự và suy đoán trước hướng rẽ nhánh, đồng thời có nhiều lõi chạy song song.

Ví dụ này cho thấy rõ vai trò của ISA như một giao ước ổn định: phần cứng bên dưới thay đổi tận gốc trong hơn bốn thập kỷ nhưng giao ước với phần mềm vẫn được giữ.

Cần nói thêm rằng ranh giới giữa ba mức không phải lúc nào cũng rạch ròi. Chẳng hạn số lượng thanh ghi khả kiến là vấn đề của ISA, nhưng số thanh ghi vật lý thực sự có trong chip lại là vấn đề của tổ chức, và hai con số này thường khác nhau do kỹ thuật đổi tên thanh ghi. Tuy vậy cách phân mức trên vẫn rất hữu ích, và trong môn học này chúng ta chủ yếu làm việc ở hai mức đầu.

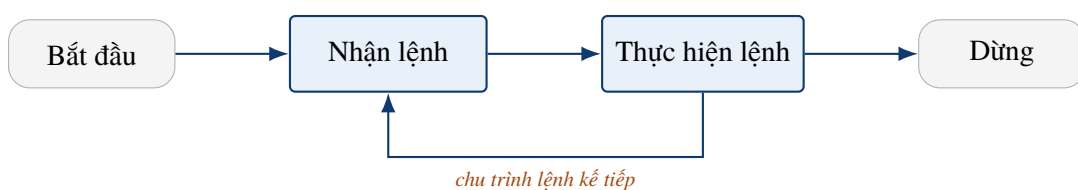
1.3 Hoạt động cơ bản của máy tính

Mọi việc mà máy tính làm đều quy về ba hoạt động: thực hiện chương trình, đáp ứng ngắt và trao đổi dữ liệu vào–ra. Mục này trình bày lần lượt từng hoạt động.

1.3.1 Thực hiện chương trình

Thực hiện chương trình là hoạt động cơ bản nhất. Bộ xử lý lặp đi lặp lại một chu trình gồm hai bước, gọi là **chu trình lệnh** (*instruction cycle*):

1. **Nhận lệnh** (*fetch*): đọc lệnh kế tiếp từ bộ nhớ vào bộ xử lý.
2. **Thực hiện lệnh** (*execute*): giải mã và thi hành thao tác mà lệnh yêu cầu.



Hình 1.3. Chu trình lệnh cơ bản

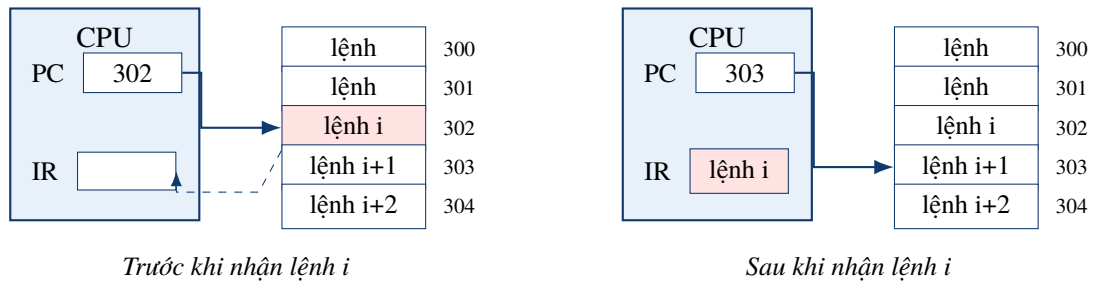
Chu trình này lặp lại liên tục từ lúc bật máy cho tới khi gặp một trong ba tình huống: lệnh đang thực hiện gây lỗi không khắc phục được, chương trình gặp lệnh dừng, hoặc người dùng tắt máy.

Bước nhận lệnh

Bắt đầu mỗi chu trình lệnh, CPU nhận lệnh từ bộ nhớ chính. Diễn biến cụ thể như sau. Bộ đếm chương trình PC (*Program Counter*) là thanh ghi giữ địa chỉ của lệnh sẽ được nhận vào. CPU

phát nội dung của PC ra bus địa chỉ để tìm ngăn nhớ chứa lệnh, đồng thời phát tín hiệu đọc bộ nhớ. Lệnh đọc được từ bộ nhớ đi theo bus dữ liệu vào thanh ghi lệnh IR (*Instruction Register*). Sau khi lệnh đã nằm trong IR, nội dung PC tự động tăng lên để trở tới lệnh kế tiếp.

Lượng tăng của PC bằng đúng độ dài của lệnh vừa nhận. Ở các kiến trúc có lệnh dài cố định như MIPS hay ARM ở chế độ 32 bit, PC luôn tăng thêm 4 (bốn byte). Ở các kiến trúc có lệnh dài thay đổi như x86, lượng tăng phụ thuộc vào lệnh cụ thể, có thể từ một tới mười lăm byte.



Hình 1.4. Trạng thái của PC, IR và bộ nhớ trước và sau bước nhận lệnh

Bước thực hiện lệnh

Bộ xử lý giải mã lệnh vừa nhận và phát các tín hiệu điều khiển tương ứng để thực hiện thao tác mà lệnh yêu cầu. Dù tập lệnh của các máy rất phong phú, mọi lệnh đều quy về bốn nhóm thao tác cơ bản:

- **Trao đổi dữ liệu giữa CPU với bộ nhớ chính**, ví dụ nạp một toán hạng từ bộ nhớ vào thanh ghi hoặc cất kết quả từ thanh ghi ra bộ nhớ.
- **Trao đổi dữ liệu giữa CPU với mô-đun vào-ra**, tức đọc dữ liệu từ một cổng hoặc ghi dữ liệu ra một cổng.
- **Thực hiện phép toán số học hoặc phép toán logic** trên dữ liệu.
- **Chuyển điều khiển trong chương trình**, tức thay đổi nội dung PC để rẽ nhánh hoặc nhảy tới một vị trí khác thay vì thực hiện lệnh kế tiếp.

Một lệnh cụ thể có thể là tổ hợp của vài thao tác trên. Lệnh cộng nội dung một ngăn nhớ vào thanh ghi chẳng hạn, vừa có thao tác trao đổi dữ liệu với bộ nhớ vừa có thao tác số học.

Ví dụ 1.3: Theo dõi việc thực hiện một chương trình

Xét một máy tính giả định có từ nhớ dài 16 bit, một thanh ghi tích lũy AC và tập lệnh rút gọn. Mỗi lệnh chiếm một từ nhớ, gồm 4 bit mã lệnh và 12 bit địa chỉ. Ta viết nội dung từ nhớ dưới dạng bốn chữ số hệ mười sáu. Ba mã lệnh cần dùng:

- | | |
|-----------|--|
| 1 – LOAD | nạp nội dung ngăn nhớ vào AC: $AC \leftarrow M(X)$ |
| 2 – STORE | cất nội dung AC ra ngăn nhớ: $M(X) \leftarrow AC$ |
| 5 – ADD | cộng nội dung ngăn nhớ vào AC: $AC \leftarrow AC + M(X)$ |

Bộ nhớ được nạp sẵn như sau (địa chỉ viết theo hệ mười sáu):

$$\begin{aligned} M(300) &= 1940, & M(301) &= 5941, & M(302) &= 2941, \\ M(940) &= 0003, & M(941) &= 0002. \end{aligned}$$

Ban đầu $PC = 300$. Hãy theo dõi diễn biến của chương trình.

Giải. Chương trình thực hiện phép cộng nội dung hai ngăn nhớ 940 và 941 rồi cất kết quả vào 941. Diễn biến qua ba chu trình lệnh:

Bước	Pha	PC	IR	AC	Diễn giải
1	Nhận lệnh	$300 \rightarrow 301$	1940	–	Đọc lệnh tại 300
2	Thực hiện	301	1940	0003	$AC \leftarrow M(940) = 3$
3	Nhận lệnh	$301 \rightarrow 302$	5941	0003	Đọc lệnh tại 301
4	Thực hiện	302	5941	0005	$AC \leftarrow 3 + M(941) = 3 + 2$
5	Nhận lệnh	$302 \rightarrow 303$	2941	0005	Đọc lệnh tại 302
6	Thực hiện	303	2941	0005	$M(941) \leftarrow 5$

Kết thúc chương trình, ngăn nhớ 941 chứa giá trị 0005. Cần chú ý hai điểm. Thứ nhất, PC luôn được tăng ngay trong pha nhận lệnh chứ không phải sau khi thực hiện xong lệnh. Thứ hai, để cộng hai số cần tới ba lệnh, trong đó có hai lần truy nhập bộ nhớ chỉ để chuyển dữ liệu. Đây chính là biểu hiện cụ thể của điểm nghẽn von Neumann sẽ nói ở mục 1.5.1.

1.3.2 Hoạt động ngắt

Định nghĩa 1.2: Ngắt

Ngắt (*interrupt*) là cơ chế cho phép CPU tạm dừng chương trình đang thực hiện để chuyển sang thực hiện một chương trình con có sẵn trong bộ nhớ, gọi là chương trình con xử lý ngắt (*interrupt handler*), rồi sau đó quay về thực hiện tiếp chương trình bị tạm dừng.

Lý do phải có ngắt nằm ở sự chênh lệch tốc độ rất lớn giữa CPU và thiết bị ngoại vi. Một bộ xử lý hiện đại thực hiện được hàng tỷ lệnh mỗi giây, trong khi một lần bấm phím hay một thao tác đọc đĩa mất hàng phần nghìn giây. Nếu CPU phải đứng chờ thiết bị bằng cách liên tục hỏi trạng thái thì phần lớn năng lực tính toán bị lãng phí. Với cơ chế ngắt, CPU cứ tiếp tục làm việc khác; khi nào thiết bị sẵn sàng thì tự nó phát tín hiệu báo cho CPU biết.

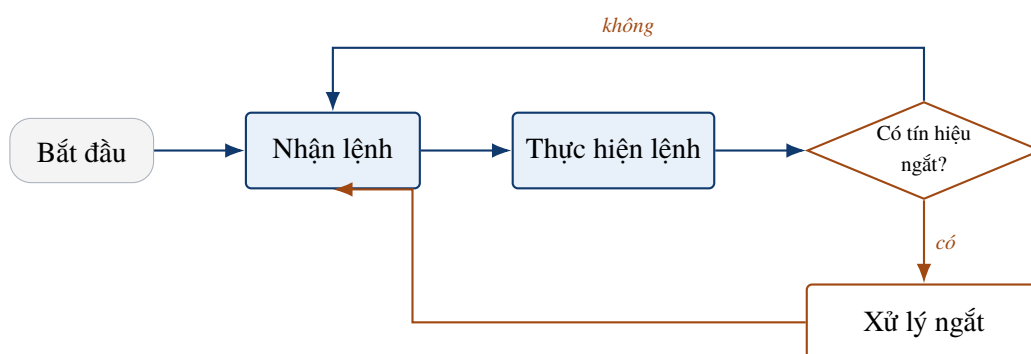
Phân loại ngắt

- **Ngắt do chương trình:** sinh ra từ chính việc thực hiện lệnh, ví dụ tràn số (*overflow*), chia cho không, thực hiện mã lệnh không hợp lệ, truy nhập vùng nhớ không được phép.
- **Ngắt từ bên ngoài (*external interrupt*):** do thiết bị vào–ra, thông qua mô–đun vào–ra, gửi tín hiệu ngắt đến CPU để yêu cầu trao đổi dữ liệu hoặc báo hoàn thành một thao tác.
- **Ngắt định thời (*timer interrupt*):** do bộ định thời bên trong bộ xử lý sinh ra theo chu kỳ. Đây là cơ sở để hệ điều hành chia thời gian cho các tiến trình.
- **Ngắt do lỗi phần cứng:** lỗi chẵn lẻ bộ nhớ, sụt nguồn, quá nhiệt.

Chu trình lệnh khi có ngắt

Sau khi hoàn thành mỗi lệnh, bộ xử lý dành một khoảng thời gian rất ngắn để kiểm tra tín hiệu ngắt. Nếu không có ngắt, nó nhận lệnh tiếp theo của chương trình hiện tại như bình thường. Nếu có tín hiệu ngắt, trình tự xử lý gồm các bước:

1. Tạm dừng (*suspend*) chương trình đang thực hiện.
2. Cất các thông tin liên quan tới chương trình bị ngắt, tối thiểu là nội dung PC và thanh ghi trạng thái, thông thường cất vào ngăn xếp.
3. Thiết lập PC trở tới địa chỉ đầu của chương trình con xử lý ngắt tương ứng, địa chỉ này lấy từ bảng vectơ ngắt.
4. Chuyển sang thực hiện chương trình con xử lý ngắt.
5. Khôi phục các thông tin đã cất và trở về thực hiện tiếp chương trình đang bị tạm dừng, tại đúng chỗ đã dừng.



Hình 1.5. Chu trình lệnh có xét tín hiệu ngắt

Chương trình bị ngắt hoàn toàn không biết là mình đã bị ngắt. Sau khi được khôi phục, nó chạy tiếp như chưa có chuyện gì xảy ra. Muốn được như vậy, chương trình con xử lý ngắt

phải trả lại nguyên vẹn mọi thanh ghi mà nó có dùng đến. Đây là yêu cầu bắt buộc khi lập trình xử lý ngắt và cũng là chỗ sinh lỗi thường gặp nhất.

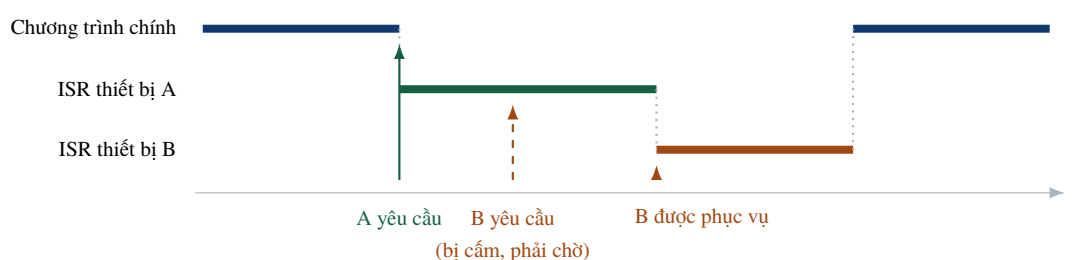
Xử lý khi có nhiều yêu cầu ngắt

Trong thực tế nhiều thiết bị có thể yêu cầu ngắt gần như đồng thời. Có hai cách giải quyết.

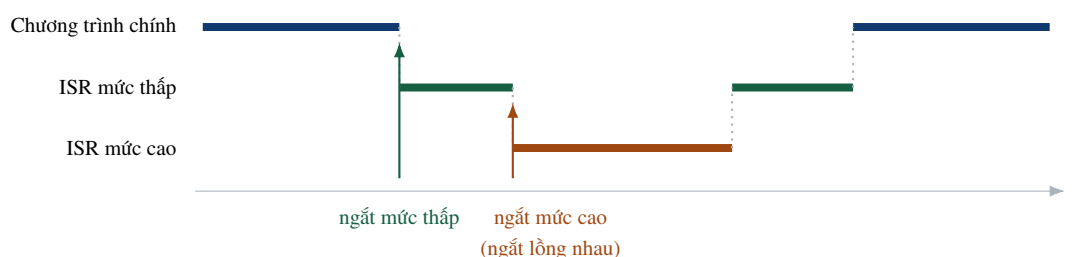
Xử lý ngắt tuần tự. Khi một ngắt đang được xử lý, bộ xử lý cấm mọi ngắt khác (*disabled interrupt*). Các yêu cầu ngắt đến sau bị bỏ qua tạm thời nhưng vẫn được ghi nhận và chờ; chúng được kiểm tra lại sau khi ngắt hiện tại đã xử lý xong. Cách này đơn giản, dễ cài đặt, nhưng không tính tới mức độ khẩn cấp: một yêu cầu rất gấp có thể phải chờ một yêu cầu không quan trọng.

Xử lý ngắt theo mức ưu tiên. Mỗi ngắt được gán một mức ưu tiên. Một ngắt đang xử lý có thể bị ngắt tiếp bởi ngắt có mức ưu tiên cao hơn, sinh ra hiện tượng ngắt lồng nhau (*nested interrupt*). Ngắt có mức ưu tiên thấp hơn hoặc bằng thì phải chờ. Cách này đáp ứng tốt yêu cầu thời gian thực nhưng phức tạp hơn và đòi hỏi ngăn xếp đủ sâu cho các mức lồng nhau.

(a) Xử lý ngắt tuần tự



(b) Xử lý ngắt theo mức ưu tiên



Hình 1.6. So sánh hai chiến lược xử lý khi có nhiều yêu cầu ngắt

Ví dụ 1.4: Lợi ích của cơ chế ngắt

Một máy tính có CPU thực hiện trung bình 10^9 lệnh mỗi giây. Chương trình cần in ra máy in một khối 1000 ký tự, máy in nhận được 100 ký tự mỗi giây. Hãy so sánh thời gian CPU bị lãng phí trong hai trường hợp: chờ bằng vòng lặp hỏi trạng thái, và dùng ngắt.

Giải. Thời gian in hết khối là $1000/100 = 10$ giây.

Nếu CPU chờ bằng vòng lặp hỏi trạng thái, suốt 10 giây đó nó không làm được việc gì khác,

tương ứng $10 \times 10^9 = 10^{10}$ lệnh bị bỏ phí.

Nếu dùng ngắt, CPU chỉ tham gia vào các thời điểm máy in báo sẵn sàng nhận ký tự tiếp theo, tức 1000 lần. Giả sử mỗi lần xử lý ngắt tốn 500 lệnh, tổng chi phí là $1000 \times 500 = 5 \times 10^5$ lệnh, chiếm 0,005% năng lực CPU trong 10 giây đó. Phần còn lại CPU dành cho các chương trình khác.

Tỷ lệ chênh lệch khoảng 20 000 lần đã giải thích vì sao mọi máy tính đa nhiệm đều phải dựa trên cơ chế ngắt.

1.3.3 Hoạt động vào–ra

Hoạt động vào–ra là hoạt động trao đổi dữ liệu giữa mô–đun vào–ra với bên trong máy tính. Có ba phương pháp tổ chức.

Vào–ra bằng chương trình (*programmed I/O*). CPU trực tiếp trao đổi dữ liệu với mô–đun vào–ra bằng các lệnh vào–ra nằm trong chương trình. CPU phải đọc thanh ghi trạng thái của mô–đun và lặp lại việc đọc cho tới khi thiết bị sẵn sàng, sau đó mới chuyển dữ liệu. Cách này đơn giản nhất nhưng lãng phí thời gian CPU nhiều nhất.

Vào–ra điều khiển bằng ngắt (*interrupt-driven I/O*). CPU phát lệnh khởi động thao tác vào–ra rồi quay về làm việc khác. Khi thiết bị sẵn sàng, mô–đun vào–ra phát tín hiệu ngắt, CPU tạm dừng để chuyển một đơn vị dữ liệu rồi lại tiếp tục công việc cũ. Cách này tiết kiệm được thời gian chờ, nhưng mỗi đơn vị dữ liệu vẫn phải đi qua CPU.

Truy nhập trực tiếp bộ nhớ (*DMA – Direct Memory Access*). CPU trao quyền điều khiển bus cho một bộ điều khiển chuyên dụng để mô–đun vào–ra trao đổi dữ liệu trực tiếp với bộ nhớ chính, không qua CPU. CPU chỉ tham gia hai lần: lúc đầu để thiết lập thông số (địa chỉ vùng nhớ, số lượng dữ liệu, chiều truyền) và lúc cuối khi nhận ngắt báo đã truyền xong. Đây là phương pháp bắt buộc phải dùng khi truyền khối lượng dữ liệu lớn, chẳng hạn đọc ghi ổ cứng hay nhận gói tin mạng.

Bảng 1.3. So sánh ba phương pháp vào–ra

	Bằng chương trình	Bằng ngắt	DMA
CPU chờ thiết bị	Có, chờ tích cực	Không	Không
Dữ liệu đi qua CPU	Có	Có	Không
Chi phí CPU cho mỗi đơn vị dữ liệu	Rất cao	Trung bình	Gần bằng không
Phần cứng phụ trợ	Không cần	Cần mạch ngắt	Cần bộ điều khiển DMA
Phù hợp với	Dữ liệu ít, thiết bị đơn giản	Dữ liệu vừa, nhiều thiết bị	Truyền khối lớn, tốc độ cao

1.4 Lịch sử phát triển máy tính

Lịch sử máy tính điện tử thường được chia thành bốn thế hệ, lấy công nghệ chế tạo linh kiện làm mốc phân chia. Cách chia này tuy có phần quy ước nhưng phản ánh khá đúng những bước nhảy về chất của ngành.

Bảng 1.4. Bốn thế hệ máy tính điện tử

Thế hệ	Thời gian	Linh kiện	Máy tiêu biểu và đặc điểm
Thứ nhất	1945 – 1955	Đèn điện tử chân không	ENIAC, IAS. Máy rất lớn, tiêu thụ điện khổng lồ, độ tin cậy thấp; lập trình bằng cách đấu dây hoặc bằng mã máy
Thứ hai	1955 – 1965	Tranzito	IBM dòng 7000, DEC PDP-1. Máy nhỏ và tin cậy hơn hẳn; xuất hiện ngôn ngữ bậc cao FORTRAN, COBOL
Thứ ba	1965 – 1980	Vi mạchSSI, MSI, LSI	IBM System/360, PDP-8. Ra đời khái niệm họ máy tính tương thích và khái niệm bus; PDP-8 là máy tính mini đầu tiên
Thứ tư	1980 – nay	Vi mạch LSI, VLSI, ULSI	Máy tính cá nhân, bộ vi xử lý, siêu máy tính, trung tâm dữ liệu, điện toán đám mây

1.4.1 Thế hệ thứ nhất: đèn điện tử chân không

Máy tính ENIAC

ENIAC (*Electronic Numerical Integrator and Computer*) là máy tính điện tử số đa dụng đầu tiên. Máy do Giáo sư John Mauchly và học trò của ông là J. Presper Eckert tại Đại học Pennsylvania thiết kế từ năm 1943 và hoàn thành năm 1946, ban đầu nhằm phục vụ việc tính bảng đạn đạo cho quân đội Mỹ.

Về quy mô, ENIAC là một cỗ máy khổng lồ: dài khoảng 20 mét, cao 2,8 mét, rộng vài mét, chiếm gần trọn một gian phòng lớn. Máy dùng 18 000 đèn điện tử, 1 500 công tắc tự động, nặng khoảng 30 tấn và tiêu thụ 140 kW. Bộ nhớ chỉ gồm 20 thanh ghi 10 chữ số, và điều đáng chú ý là máy tính toán trực tiếp trên hệ thập phân chứ không phải hệ nhị phân. Tốc độ đạt khoảng 5 000 phép cộng mỗi giây, nhanh hơn máy cơ khí đương thời hàng nghìn lần.

Nhược điểm lớn nhất của ENIAC nằm ở khâu lập trình. Muốn đổi bài toán, người ta phải đấu lại các đầu cắm điện và đặt lại hàng loạt ngắt điện bằng tay, công việc này có khi mất vài ngày. Chính hạn chế đó đã thúc đẩy sự ra đời của ý tưởng chương trình lưu trữ trong bộ nhớ.

Máy tính IAS

Máy IAS được thực hiện tại Viện Nghiên cứu Cao cấp Princeton (*Institute for Advanced Studies*), do John von Neumann thiết kế theo ý tưởng chương trình lưu trữ (*stored program*). Khác với ENIAC, IAS xử lý theo hệ nhị phân. Cấu trúc của IAS trở thành mô hình cơ bản cho hầu hết máy tính về sau, nên sẽ được trình bày kỹ ở mục 1.5.1.

1.4.2 Thế hệ thứ hai: tranzito

Tranzito được phát minh tại phòng thí nghiệm Bell năm 1947 và bắt đầu thay thế đèn điện tử trong máy tính từ cuối những năm 1950. So với đèn, tranzito nhỏ hơn, rẻ hơn, tỏa nhiệt ít hơn và bền hơn rất nhiều. Nhờ đó máy tính thế hệ này có kích thước và giá thành giảm mạnh trong khi độ tin cậy tăng lên, mở đường cho việc thương mại hóa. IBM với dòng máy lớn 7000 series là đại diện tiêu biểu.

Cùng thời kỳ này xuất hiện các ngôn ngữ lập trình bậc cao đầu tiên và khái niệm hệ điều hành sơ khai dưới dạng chương trình giám sát xử lý theo lô.

1.4.3 Thế hệ thứ ba: vi mạch tích hợp

Việc chế tạo được nhiều tranzito trên cùng một phiến bán dẫn đã sinh ra vi mạch tích hợp (IC). Mức độ tích hợp tăng dần qua các cấp SSI, MSI rồi LSI, kéo theo kích thước và giá thành máy tính tiếp tục giảm.

Hai sản phẩm mang tính bước ngoặt của thế hệ này là IBM System/360 và DEC PDP-8. System/360 lần đầu tiên đưa ra khái niệm *họ máy tính*: nhiều mẫu máy có năng lực và giá khác nhau nhưng dùng chung một kiến trúc tập lệnh, nhờ đó phần mềm chạy được trên cả họ. PDP-8 là máy tính mini đầu tiên, giá chỉ khoảng 16 000 đô-la, và quan trọng hơn về mặt kiến trúc, nó dùng cấu trúc bus chung thay cho cách nối điểm nối điểm trước đó.

1.4.4 Thế hệ thứ tư: vi mạch cỡ lớn và bộ vi xử lý

Từ năm 1980 tới nay, công nghệ LSI, VLSI rồi ULSI cho phép đặt toàn bộ một bộ xử lý lên một chip duy nhất, tạo ra bộ vi xử lý (*microprocessor*). Đây là điều kiện trực tiếp để máy tính cá nhân ra đời và phổ cập. Ở đầu kia của phổ, cũng chính công nghệ đó cho phép ghép hàng chục nghìn bộ xử lý thành siêu máy tính và trung tâm dữ liệu, làm nền cho điện toán đám mây ngày nay.

Bảng 1.5. Các mốc phát triển của bộ vi xử lý

Năm	Sự kiện
1971	Intel 4004, bộ vi xử lý đầu tiên, xử lý dữ liệu 4 bit
1972	Các bộ vi xử lý 8 bit ra đời
1978	Các bộ vi xử lý 16 bit; máy tính cá nhân IBM-PC ra đời năm 1981
1985	Các bộ vi xử lý 32 bit
2001	Các bộ vi xử lý 64 bit
2006	Bộ vi xử lý đa lõi, nhiều lõi xử lý trên cùng một chip

Mốc chuyển sang đa lõi từ năm 2006 đánh dấu một thay đổi về chất trong tư duy thiết kế. Trước đó, cách chủ yếu để tăng hiệu năng là tăng tần số xung nhịp. Khi tần số chạm ngưỡng vài GHz thì công suất tiêu thụ và nhiệt lượng tỏa ra trở nên không thể xử lý được nữa, buộc các nhà thiết kế chuyển hướng sang tăng số lõi. Hệ quả là gánh nặng khai thác hiệu năng chuyển một phần đáng kể sang phía người lập trình: một chương trình đơn luồng ngày nay hầu như không còn tự nhanh lên khi đổi bộ xử lý mới.

1.5 Một số kiến trúc máy tính cơ bản

1.5.1 Kiến trúc von Neumann

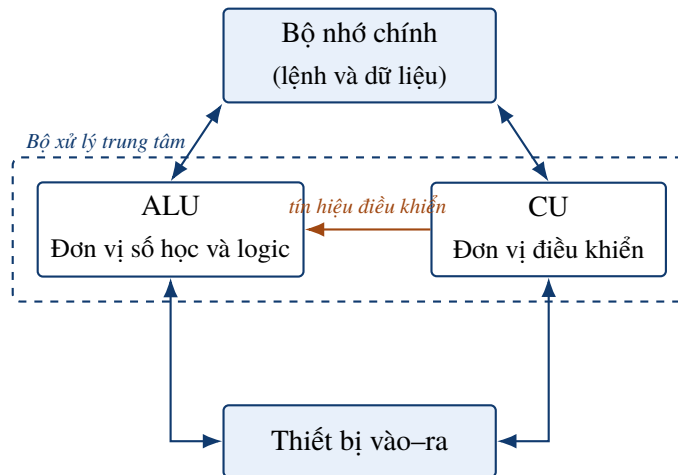
Như đã nói, ENIAC là máy tính đa dụng đầu tiên nhưng nặng nề và bất tiện vì phải lập trình bằng cách đấu dây. John von Neumann đề xuất một kiến trúc mới, hiện thực hóa trong máy IAS, và kiến trúc đó trở thành khuôn mẫu cho hầu hết máy tính đa dụng về sau.

BA NGUYÊN LÝ CỦA CHƯƠNG TRÌNH LƯU TRỮ

1. Dữ liệu và lệnh cùng được lưu trữ trong một bộ nhớ đọc ghi duy nhất.
2. Nội dung bộ nhớ được định vị theo vị trí, tức theo địa chỉ, không phụ thuộc vào việc nội dung đó là dữ liệu hay là lệnh.
3. Chương trình được thực thi tuần tự từ lệnh này sang lệnh kế tiếp, trừ khi có lệnh chuyển điều khiển làm thay đổi trật tự đó.

Nguyên lý thứ nhất và thứ hai gộp lại có một hệ quả sâu xa: vì lệnh cũng chỉ là các dãy bit nằm trong bộ nhớ như mọi dữ liệu khác, nên một chương trình hoàn toàn có thể đọc và sửa chính mình hoặc sinh ra chương trình khác. Đây là nền tảng của trình biên dịch, trình nạp, máy ảo và mọi công cụ phần mềm hiện đại. Mặt trái của nó là các lỗ hổng bảo mật kiểu tràn bộ đệm, khi dữ liệu do kẻ tấn công đưa vào được máy hiểu nhầm thành lệnh.

Cấu trúc máy tính IAS



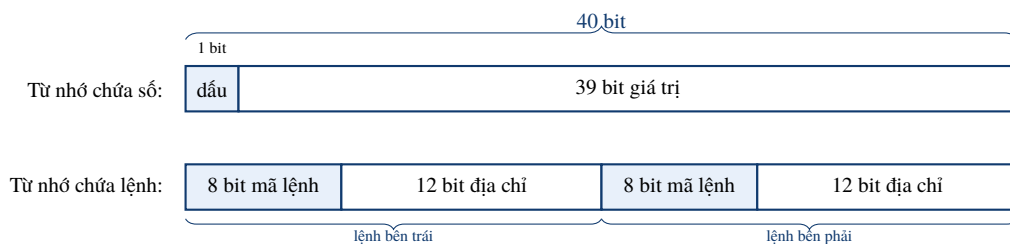
Hình 1.7. Cấu trúc tổng quát của máy tính IAS

Bốn thành phần của IAS đảm nhiệm các vai trò sau. Bộ nhớ chính lưu trữ cả lệnh lẫn dữ liệu. Đơn vị số học và logic ALU xử lý trên dữ liệu nhị phân. Đơn vị điều khiển CU điều hành quá trình thực thi lệnh bằng cách lấy lệnh từ bộ nhớ, giải mã và phát tín hiệu điều khiển. Thiết bị vào-ra nhận sự điều khiển từ chương trình thông qua CU.

Tổ chức bộ nhớ và khuôn dạng lệnh của IAS

Bộ nhớ IAS chứa được 1000 vị trí, gọi là từ nhớ (*word*), mỗi từ nhớ dài 40 bit. Một từ nhớ có thể được dùng theo hai cách:

- Chứa **một số**: 1 bit dấu và 39 bit giá trị.
- Chứa **hai lệnh**, mỗi lệnh dài 20 bit gồm 8 bit mã lệnh (*opcode*) và 12 bit địa chỉ.



Hình 1.8. Hai cách sử dụng một từ nhớ 40 bit trong máy IAS

Ví dụ 1.5: Kiểm tra tính hợp lý của trường địa chỉ IAS

Trường địa chỉ trong lệnh của IAS dài 12 bit. Hãy kiểm tra xem độ dài này có đủ để đánh địa chỉ toàn bộ bộ nhớ 1000 từ hay không, và cho biết còn dư bao nhiêu.

Giải. Với 12 bit, số địa chỉ phân biệt được là $2^{12} = 4096$. Bộ nhớ chỉ có 1000 từ, nhỏ hơn

4096, nên 12 bit hoàn toàn đủ; phần dư là $4096 - 1000 = 3096$ địa chỉ chưa dùng đến. Phần dư này không phải là lãng phí vô ích: nó để ngỏ khả năng mở rộng dung lượng bộ nhớ về sau mà không phải thay đổi khuôn dạng lệnh, tức không phá vỡ tính tương thích của phần mềm. Việc chứa sẵn không gian địa chỉ như vậy là một quyết định thiết kế thường gặp và ta còn gặp lại nó ở nhiều kiến trúc sau này.

Tập thanh ghi của IAS

Cả ALU và CU đều có các đơn vị lưu trữ hỗ trợ gọi là thanh ghi (*register*).

Bảng 1.6. Các thanh ghi của máy IAS

Ký hiệu	Tên đầy đủ	Chức năng
MBR	Memory Buffer Register	Đệm dữ liệu hoặc lệnh đọc từ bộ nhớ, hoặc dữ liệu chuẩn bị ghi vào bộ nhớ
MAR	Memory Address Register	Giữ địa chỉ của ngăn nhớ sắp được truy nhập
IR	Instruction Register	Giữ mã lệnh 8 bit của lệnh đang được thực hiện
IBR	Instruction Buffer Register	Đệm tạm lệnh bên phải của từ nhớ vừa đọc, để lần sau khỏi phải đọc lại bộ nhớ
PC	Program Counter	Giữ địa chỉ của cặp lệnh sẽ được nhận vào tiếp theo
AC, MQ	Accumulator, Multiplier/Quotient	Lưu tạm toán hạng và kết quả của ALU

Thanh ghi IBR đáng chú ý vì nó cho thấy tư duy tối ưu hóa đã xuất hiện rất sớm. Do mỗi từ nhớ chứa hai lệnh, nếu không có IBR thì bộ xử lý phải đọc bộ nhớ hai lần cho cùng một từ. IBR giữ lại lệnh bên phải sau lần đọc đầu, nhờ đó tiết kiệm được một nửa số lần truy nhập bộ nhớ khi lấy lệnh. Về bản chất, đây là một dạng sơ khai của cache lệnh.

Điểm nghẽn von Neumann

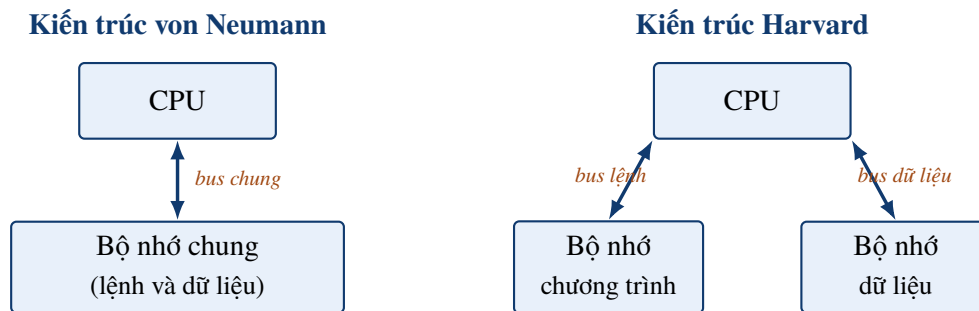
Kiến trúc von Neumann dùng chung một bộ nhớ và một hệ thống bus cho cả lệnh lẫn dữ liệu. Ưu điểm là đơn giản và mềm dẻo, nhưng nhược điểm cố hữu là bộ xử lý không thể đọc lệnh và đọc dữ liệu cùng lúc, vì cả hai đều phải đi qua một con đường duy nhất. Khi tốc độ CPU tăng nhanh hơn nhiều so với tốc độ bộ nhớ, con đường chung này trở thành nút cổ chai giới hạn hiệu năng toàn hệ thống, gọi là **điểm nghẽn von Neumann** (*von Neumann bottleneck*).

Các kỹ thuật như phân cấp bộ nhớ nhiều mức cache, đường ống, tách cache lệnh và cache dữ liệu đều là những cách khắc phục hoặc làm nhẹ điểm nghẽn này.

1.5.2 Kiến trúc Harvard

Kiến trúc Harvard tách riêng bộ nhớ chứa chương trình và bộ nhớ chứa dữ liệu, mỗi bộ nhớ có hệ thống bus riêng nối tới CPU. Nhờ vậy CPU truy xuất được đồng thời cả hai bộ nhớ trong cùng một chu kỳ.

Vì hai bus tách rời nhau, độ rộng của chúng không cần bằng nhau. Bus chương trình được chọn đúng bằng độ dài một lệnh, thay đổi linh hoạt và tối ưu theo từng dòng thiết bị, chẳng hạn 12, 14 hay 16 bit ở các họ vi điều khiển. Bus dữ liệu thường là 8 hoặc 16 bit theo độ dài dữ liệu cần xử lý.



Hình 1.9. So sánh sơ đồ nguyên lý của hai kiến trúc

Ưu điểm của kiến trúc Harvard

- Mọi lệnh đều chiếm đúng một vị trí nhớ, do bus chương trình được thiết kế vừa khít với độ dài lệnh. Việc giải mã và tính địa chỉ lệnh nhờ đó rất đơn giản.
- Tốc độ xử lý tăng vì mã lệnh và địa chỉ toán hạng liên quan cùng nằm gọn trong một từ.
- Chương trình chạy nhanh hơn vì có thể nhận lệnh kế tiếp trong lúc vẫn đang đọc hoặc ghi dữ liệu của lệnh hiện tại.

Nhược điểm và kiến trúc Harvard cải tiến

Nhược điểm của Harvard thuần túy là kém mềm dẻo. Vùng nhớ chương trình và vùng nhớ dữ liệu có dung lượng cố định, không san sẻ cho nhau được; chương trình cũng khó tự đọc dữ liệu đặt trong bộ nhớ mã, và việc nạp chương trình mới cần cơ chế riêng. Ngoài ra chi phí phần cứng cao hơn do phải có hai hệ thống bus.

Vì lý do đó, hầu hết bộ xử lý đa dụng hiện nay dùng **kiến trúc Harvard cải tiến** (*modified Harvard*): ở mức bộ nhớ chính vẫn là von Neumann với một không gian địa chỉ thống nhất, nhưng ở mức cache trong CPU thì tách riêng cache lệnh L1I và cache dữ liệu L1D. Cách làm này giữ được tính mềm dẻo của von Neumann ở bên ngoài, đồng thời hưởng lợi ích song song của Harvard ở bên trong, nơi quyết định hiệu năng.

Bảng 1.7. So sánh kiến trúc von Neumann và kiến trúc Harvard

Tiêu chí	von Neumann	Harvard
Bộ nhớ	Chung cho lệnh và dữ liệu	Tách riêng hai bộ nhớ
Hệ thống bus	Một bus chung	Hai bus độc lập
Truy xuất đồng thời	Không	Có
Độ rộng bus	Lệnh và dữ liệu dùng chung độ rộng	Có thể khác nhau, tối ưu riêng
Tính mềm dẻo	Cao, phân chia vùng nhớ tùy ý	Thấp, dung lượng hai vùng cố định
Chi phí phần cứng	Thấp hơn	Cao hơn
Ứng dụng điển hình	Máy tính đa dụng	Vi điều khiển, bộ xử lý tín hiệu số

1.6 Các hệ đếm và tổ chức lưu trữ dữ liệu trên máy tính

Ba hệ đếm được dùng thường xuyên khi làm việc với máy tính, mỗi hệ phục vụ một mục đích riêng:

- **Hệ thập phân** (*decimal*, cơ số 10): hệ mà con người quen dùng.
- **Hệ nhị phân** (*binary*, cơ số 2): hệ mà máy tính thực sự dùng, vì mạch điện tử chỉ phân biệt tin cậy hai mức điện áp.
- **Hệ thập lục phân** (*hexadecimal*, cơ số 16): dùng để viết gọn số nhị phân cho người đọc, không phải là hệ mà máy dùng.

1.6.1 Nguyên tắc chung của hệ đếm theo vị trí

Trong một hệ đếm theo vị trí cơ số b , mỗi chữ số có giá trị phụ thuộc vào vị trí của nó. Số S gồm các chữ số $C_n C_{n-1} \dots C_1 C_0$ có giá trị:

$$S_b = C_n b^n + C_{n-1} b^{n-1} + \dots + C_1 b^1 + C_0 b^0 = \sum_{i=0}^n C_i b^i,$$

trong đó mỗi chữ số C_i thỏa mãn $0 \leq C_i \leq b - 1$. Đại lượng b^i gọi là *trọng số* của vị trí thứ i . Trọng số tăng dần từ phải qua trái.

1.6.2 Hệ thập phân

Hệ thập phân có cơ số 10, dùng mười chữ số từ 0 tới 9. Các cột từ phải qua trái lần lượt là hàng đơn vị, hàng chục, hàng trăm và cứ thế tiếp tục.

Ví dụ 1.6: Khai triển một số thập phân

$$(123)_{10} = 1 \times 10^2 + 2 \times 10^1 + 3 \times 10^0 = 100 + 20 + 3 = 123.$$

Ta sẽ dùng lại chính số 123 này trong các ví dụ tiếp theo để tiện đối chiếu giữa các hệ đếm.

1.6.3 Hệ nhị phân

Hệ nhị phân có cơ số 2. Mỗi chữ số nhị phân gọi là một **bit** (*binary digit*) và chỉ nhận một trong hai giá trị 0 hoặc 1. Giá trị của một số nhị phân $n+1$ bit:

$$S_2 = \sum_{i=0}^n C_i 2^i.$$

Hai bit ở hai đầu có tên riêng vì hay được nhắc tới:

- **MSB** (*Most Significant Bit*): bit có trọng số lớn nhất, nằm ngoài cùng bên trái.
- **LSB** (*Least Significant Bit*): bit có trọng số nhỏ nhất, nằm ngoài cùng bên phải.

Đổi nhị phân sang thập phân

Chỉ việc khai triển theo trọng số rồi cộng lại.

Ví dụ 1.7: Đổi nhị phân sang thập phân

Đổi $(1111011)_2$ sang hệ thập phân.

Giải. Viết trọng số dưới từng bit rồi cộng các trọng số ứng với bit 1:

1	1	1	1	0	1	1
2^6	2^5	2^4	2^3	2^2	2^1	2^0
64	32	16	8	4	2	1

$$(1111011)_2 = 64 + 32 + 16 + 8 + 0 + 2 + 1 = (123)_{10}.$$

Đổi thập phân sang nhị phân

Có hai cách làm, nên biết cả hai vì mỗi cách tiện trong một tình huống.

Cách 1: chia liên tiếp cho 2 và lấy phần dư. Chia số cần đổi cho 2, ghi lại số dư, rồi tiếp tục chia thương cho 2 cho tới khi thương bằng 0. Dãy số dư đọc *ngược từ dưới lên* chính là kết quả. Số dư đầu tiên là LSB, số dư cuối cùng là MSB. Cách này máy móc, luôn đúng, hợp với số lớn.

Cách 2: phân tích thành tổng các lũy thừa của 2. Tìm lũy thừa của 2 lớn nhất còn nhỏ hơn

hoặc bằng số đã cho, trừ đi, rồi lặp lại với phần còn lại. Cách này nhanh hơn nhiều khi làm nhầm, với điều kiện thuộc bảng lũy thừa của 2.

Ví dụ 1.8: Đổi thập phân sang nhị phân bằng hai cách

Đổi $(123)_{10}$ sang hệ nhị phân.

Cách 1 – chia liên tiếp cho 2.

Phép chia	Thương	Dư	
$123 : 2 \rightarrow$	61	1	$\leftarrow \text{LSB}$
$61 : 2 \rightarrow$	30	1	
$30 : 2 \rightarrow$	15	0	
$15 : 2 \rightarrow$	7	1	
$7 : 2 \rightarrow$	3	1	
$3 : 2 \rightarrow$	1	1	
$1 : 2 \rightarrow$	0	1	$\leftarrow \text{MSB}$

Đọc các số dư từ dưới lên: $(123)_{10} = (1111011)_2$.

Cách 2 – phân tích thành tổng lũy thừa của 2.

$$\begin{aligned}
 123 &= 64 + 59 = 64 + 32 + 27 = 64 + 32 + 16 + 11 \\
 &= 64 + 32 + 16 + 8 + 3 = 64 + 32 + 16 + 8 + 2 + 1,
 \end{aligned}$$

tức là

$$123 = 2^6 + 2^5 + 2^4 + 2^3 + 2^1 + 2^0 \Rightarrow (1111011)_2.$$

Các vị trí có mặt trong tổng ghi bit 1, vị trí vắng mặt (ở đây là 2^2) ghi bit 0.

Số nhị phân có phần lẻ

Quy tắc trọng số vẫn giữ nguyên, chỉ khác là các vị trí sau dấu phẩy có trọng số $2^{-1}, 2^{-2}, \dots$. Để đổi phần lẻ thập phân sang nhị phân, ta nhân liên tiếp phần lẻ với 2 và lấy phần nguyên của kết quả.

Ví dụ 1.9: Đổi số có phần lẻ

Đổi $(13,375)_{10}$ sang hệ nhị phân.

Giải. Tách thành phần nguyên và phần lẻ.

Phần nguyên: $13 = 8 + 4 + 1 = 2^3 + 2^2 + 2^0 \Rightarrow (1101)_2$.

Phần lẻ, nhân liên tiếp với 2:

Phép nhân	Kết quả	Phần nguyên lấy ra
$0,375 \times 2$	0,75	0
$0,75 \times 2$	1,5	1
$0,5 \times 2$	1,0	1

Đọc các phần nguyên theo thứ tự lấy ra từ trên xuống: $0,375 = (0,011)_2$. Vậy $(13,375)_{10} = (1101,011)_2$. Ta sẽ dùng lại kết quả này ở phần số dấu phẩy động.

Cần lưu ý rằng không phải phần lẻ nào cũng đổi được thành dãy bit hữu hạn. Chẳng hạn 0,1 trong hệ thập phân cho dãy nhị phân tuần hoàn vô hạn, nên máy tính buộc phải cắt bớt và sinh ra sai số. Đó là lý do trong hầu hết ngôn ngữ lập trình, phép so sánh $0.1 + 0.2 == 0.3$ cho kết quả sai.

1.6.4 Hệ thập lục phân

Hệ thập lục phân có cơ số 16, dùng mười sáu ký tự: các chữ số 0 tới 9 và các chữ cái A, B, C, D, E, F thay cho các giá trị từ 10 tới 15.

Công dụng chính của hệ này là viết gọn số nhị phân. Vì $16 = 2^4$ nên cứ một nhóm bốn bit tương ứng đúng một chữ số thập lục phân, việc chuyển đổi qua lại chỉ là tra bảng, không phải tính toán. Một địa chỉ 32 bit viết bằng nhị phân gồm 32 ký tự rất dễ nhầm, viết bằng thập lục phân chỉ còn 8 ký tự.

Bảng 1.8. Bảng tra tương ứng giữa ba hệ đếm

Thập phân	Nhị phân	Thập lục phân	Thập phân	Nhị phân	Thập lục phân
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	10	1010	A
3	0011	3	11	1011	B
4	0100	4	12	1100	C
5	0101	5	13	1101	D
6	0110	6	14	1110	E
7	0111	7	15	1111	F

Ví dụ 1.10: Chuyển đổi nhanh giữa nhị phân và thập lục phân

Nhóm bốn bit một, tính từ bên phải, rồi tra bảng 1.8:

$$\begin{aligned}(1011\ 0011)_2 &= (B3)_{16}, & (0000\ 0000)_2 &= (00)_{16}, \\ (0010\ 1101\ 1001\ 1010)_2 &= (2D9A)_{16}, & (1111\ 1111\ 1111\ 1111)_2 &= (FFFF)_{16}.\end{aligned}$$

Nếu số bit không chia hết cho 4 thì thêm các bit 0 vào bên trái cho đủ nhóm. Chẳng hạn $(1\ 0110)_2 = (0001\ 0110)_2 = (16)_{16}$.

Ví dụ 1.11: Đổi qua lại giữa thập lục phân và thập phân

a) Đổi $(7B)_{16}$ sang hệ thập phân. Khai triển theo trọng số với cơ số 16, nhớ rằng B tương ứng giá trị 11:

$$(7B)_{16} = 7 \times 16^1 + 11 \times 16^0 = 112 + 11 = (123)_{10}.$$

b) Đổi $(123)_{10}$ sang hệ thập lục phân. Chia liên tiếp cho 16 và lấy phần dư:

Phép chia	Thương	Dư
$123 : 16 \rightarrow$	7	11 (tức B) $\leftarrow$ chữ số hàng đơn vị
$7 : 16 \rightarrow$	0	7 $\leftarrow$ chữ số hàng 16^1

Đọc ngược từ dưới lên: $(123)_{10} = (7B)_{16}$.

Kết hợp ba ví dụ vừa qua ta có: $(123)_{10} = (1111011)_2 = (7B)_{16}$. Sinh viên nên tự kiểm tra lại bằng cách nhóm bốn bit: $0111\ 1011 \rightarrow 7B$.

Trong tài liệu kỹ thuật và trong mã nguồn, người ta dùng nhiều quy ước ký hiệu cơ số khác nhau. Hay gặp nhất là: $(1011)_2$ hoặc $1011B$ cho nhị phân; $(7B)_{16}$, $0 \times 7B$ (ngôn ngữ C và các ngôn ngữ dẫn xuất) hoặc $7Bh$ (hợp ngữ) cho thập lục phân. Khi đọc tài liệu cần xác định rõ quy ước đang dùng, vì dãy ký tự 10 có thể là mười, là hai hoặc là mười sáu tùy cơ số.

1.6.5 Đơn vị dữ liệu và thông tin trong máy tính

Mọi lệnh và mọi dữ liệu trong máy tính đều được mã hóa bằng số nhị phân. Các đơn vị đo lường dữ liệu như sau:

- **Bit:** đơn vị thông tin nhỏ nhất, nhận một trong hai giá trị 0 hoặc 1.
- **Nibble:** tổ hợp 4 bit, tương ứng đúng một chữ số thập lục phân.
- **Byte:** tổ hợp 8 bit, biểu diễn được $2^8 = 256$ giá trị khác nhau. Byte là đơn vị đánh địa chỉ nhỏ nhất của hầu hết máy tính hiện nay.

- **Từ** (*word*): độ dài dữ liệu tự nhiên mà bộ xử lý thao tác trong một lần, tùy kiến trúc mà bằng 16, 32 hay 64 bit.

Bảng 1.9. Quy ước các đơn vị dữ liệu bội của byte

Ký hiệu	Tên gọi	Quan hệ	Số byte
KB	Kilobyte	2^{10} byte	1 024
MB	Megabyte	2^{10} KB	$2^{20} = 1\,048\,576$
GB	Gigabyte	2^{10} MB	2^{30}
TB	Terabyte	2^{10} GB	2^{40}
PB	Petabyte	2^{10} TB	2^{50}
EB	Exabyte	2^{10} PB	2^{60}

Bảng trên dùng bội số $2^{10} = 1024$, đúng với cách tính dung lượng bộ nhớ. Tuy nhiên các nhà sản xuất ổ đĩa lại tính theo bội số 1000 của hệ SI. Vì vậy một ổ cứng ghi 1 TB trên vỏ hộp (10^{12} byte) khi đưa vào hệ điều hành chỉ hiện khoảng 931 GB (2^{40} byte); chênh lệch khoảng 7%, và càng lên đơn vị cao thì càng lệch nhiều. Để tránh nhập nhằng, tổ chức IEC đã đưa ra bộ tiền tố riêng cho lũy thừa của 2: KiB, MiB, GiB, TiB. Theo đó 1 KiB = 1024 byte còn 1 kB = 1000 byte. Trong môn học này, khi không nói gì khác, ta hiểu KB, MB, GB theo lũy thừa của 2 như bảng 1.9.

1.6.6 Biểu diễn số nguyên

Số nguyên không dấu

Dùng n bit để biểu diễn số nguyên không dấu A với dãy bit $c_{n-1}c_{n-2} \dots c_1c_0$. Giá trị của A tính theo công thức khai triển quen thuộc:

$$A = \sum_{i=0}^{n-1} c_i 2^i, \quad \text{dải biểu diễn: } A \in [0, 2^n - 1].$$

Bảng 1.10. Dải biểu diễn của số nguyên không dấu theo độ dài từ

Số bit n	Dải giá trị	Giá trị lớn nhất
8	$[0, 2^8 - 1]$	255
16	$[0, 2^{16} - 1]$	65 535
32	$[0, 2^{32} - 1]$	4 294 967 295
64	$[0, 2^{64} - 1]$	18 446 744 073 709 551 615

Ví dụ 1.12: Số nguyên không dấu

a) Biểu diễn các số nguyên không dấu $A = 41$ và $B = 150$ bằng 8 bit.

$$41 = 32 + 8 + 1 = 2^5 + 2^3 + 2^0 \Rightarrow A = 0010\ 1001.$$

$$150 = 128 + 16 + 4 + 2 = 2^7 + 2^4 + 2^2 + 2^1 \Rightarrow B = 1001\ 0110.$$

b) Cho hai số nguyên không dấu 8 bit $M = 0001\ 0010$ và $N = 1011\ 1001$. Xác định giá trị của chúng.

$$M = 2^4 + 2^1 = 16 + 2 = 18.$$

$$N = 2^7 + 2^5 + 2^4 + 2^3 + 2^0 = 128 + 32 + 16 + 8 + 1 = 185.$$

c) Nếu dùng 16, 32 hay 64 bit thì cách làm hoàn toàn không đổi, chỉ cần thêm các bit 0 vào bên trái. Chẳng hạn với 16 bit, $A = 0000\ 0000\ 0010\ 1001$.

Số bù một và số bù hai

Trước khi nói về số có dấu, cần nắm hai phép lấy bù.

- **Số bù một** của một số nhị phân là kết quả của việc nghịch đảo từng bit của nó, đổi 0 thành 1 và 1 thành 0. Về giá trị, số bù một của A bằng $(2^n - 1) - A$.
- **Số bù hai** bằng số bù một cộng thêm 1. Về giá trị, số bù hai của A bằng $2^n - A$.

Ví dụ 1.13: Lấy bù một và bù hai

Với $n = 8$ bit, xét số $A = 1$, tức $0000\ 0001$.

Số ban đầu	0000 0001
Bù một (đảo từng bit)	1111 1110
Cộng thêm 1	1
Bù hai	1111 1111

Kiểm tra bằng công thức: $2^8 - 1 = 256 - 1 = 255 = 1111\ 1111$. Đúng. Dãy bit $1111\ 1111$ chính là cách máy tính biểu diễn số -1 bằng 8 bit.

MẸO LẤY BÙ HAI NHANH

Thay vì đảo hết các bit rồi cộng 1, có thể làm trực tiếp: đi từ phải sang trái, *giữ nguyên* mọi bit 0 và bit 1 đầu tiên gặp được, sau đó *đảo* tất cả các bit còn lại.

Ví dụ với $0101\ 1000$: giữ nguyên phần đuôi 1000 , đảo phần đầu 0101 thành 1010 , kết quả là $1010\ 1000$. Cách này nhanh hơn và ít nhầm hơn khi làm bài thi.

Số nguyên có dấu theo mã bù hai

Hầu như mọi máy tính hiện nay đều biểu diễn số nguyên có dấu bằng mã bù hai. Nguyên tắc: dùng n bit biểu diễn số nguyên có dấu A với dãy bit $a_{n-1}a_{n-2} \dots a_0$, trong đó

- Nếu A là số dương: bit $a_{n-1} = 0$, các bit còn lại biểu diễn độ lớn giống như số không dấu.
- Nếu A là số âm: A được biểu diễn bởi số bù hai của số dương tương ứng, do đó bit $a_{n-1} = 1$.

Bit a_{n-1} vì vậy đóng vai trò bit dấu. Công thức tính giá trị viết gọn được thành một biểu thức chung cho cả hai trường hợp:

$$A = -a_{n-1} 2^{n-1} + \sum_{i=0}^{n-2} a_i 2^i.$$

Từ đó suy ra dải biểu diễn:

$$\text{số dương: } [0, +(2^{n-1} - 1)], \quad \text{số âm: } [-2^{n-1}, -1].$$

Với $n = 8$, dải biểu diễn là $[-128, +127]$, còn nếu coi là không dấu thì là $[0, 255]$.

Dải giá trị của mã bù hai *không đối xứng*: có 128 số âm nhưng chỉ có 127 số dương, cộng thêm số 0. Nguyên nhân là số 0 chỉ có duy nhất một cách biểu diễn là 0000 0000, khác với mã dấu – độ lớn hay mã bù một vốn có tới hai cách biểu diễn số 0. Chính tính duy nhất này, cùng với việc phép trừ quy được về phép cộng, là lý do mã bù hai được chọn. Hệ quả thực tế của tính bất đối xứng: với 8 bit, số -128 biểu diễn được nhưng $+128$ thì không, và $-(-128)$ gây tràn số.

Ví dụ 1.14: Biểu diễn số nguyên có dấu 8 bit

Biểu diễn $A = +58$ và $B = -80$ bằng 8 bit theo mã bù hai.

Với $A = +58$. Là số dương nên chỉ cần đổi 58 sang nhị phân rồi đặt bit dấu bằng 0:

$$58 = 32 + 16 + 8 + 2 = 2^5 + 2^4 + 2^3 + 2^1 \Rightarrow A = 0011\ 1010.$$

Với $B = -80$. Trước hết biểu diễn số dương tương ứng:

$$80 = 64 + 16 = 2^6 + 2^4 \Rightarrow 0101\ 0000.$$

Sau đó lấy bù hai. Dùng mẹo nhanh: giữ nguyên đuôi 10000, đảo phần còn lại 010 thành 101, được

$$B = 1011\ 0000.$$

Kiểm tra lại bằng công thức giá trị: $-128 + 32 + 16 = -80$. Đúng.

Ví dụ 1.15: Xác định giá trị của số nguyên có dấu

Cho hai số nguyên có dấu 8 bit $P = 0110\ 0010$ và $Q = 1101\ 1011$. Xác định giá trị của chúng.

Với P . Bit dấu bằng 0 nên P là số dương, đọc như số không dấu:

$$P = 2^6 + 2^5 + 2^1 = 64 + 32 + 2 = +98.$$

Với Q . Bit dấu bằng 1 nên Q là số âm. Có hai cách tính.

Cách 1, dùng công thức trực tiếp:

$$Q = -2^7 + 2^6 + 2^4 + 2^3 + 2^1 + 2^0 = -128 + 64 + 16 + 8 + 2 + 1 = -37.$$

Cách 2, lấy bù hai để tìm độ lớn: bù hai của $1101\ 1011$ là $0010\ 0101$, có giá trị $32 + 4 + 1 = 37$. Vậy $Q = -37$.

Hai cách cho cùng kết quả. Cách 2 thường ít nhầm hơn khi làm nhầm.

Mở rộng bit cho số nguyên

Khi cần chuyển một số từ dạng n bit sang dạng m bit với $m > n$, chẳng hạn gán một biến 8 bit vào biến 16 bit, phải bổ sung các bit vào bên trái. Quy tắc bổ sung phụ thuộc vào việc số đó có dấu hay không.

- **Mở rộng theo số không dấu** (*zero-extended*): thêm các bit 0 vào bên trái.
- **Mở rộng theo số có dấu** (*sign-extended*): thêm các bit bằng đúng bit dấu vào bên trái. Với số dương thì thêm bit 0, với số âm thì thêm bit 1.

Ví dụ 1.16: Mở rộng bit

$$+19 = 0001\ 0011\ (8\ \text{bit}) \quad \rightarrow \quad 0000\ 0000\ 0001\ 0011\ (16\ \text{bit})$$

$$-19 = 1110\ 1101\ (8\ \text{bit}) \quad \rightarrow \quad 1111\ 1111\ 1110\ 1101\ (16\ \text{bit})$$

Kiểm tra trường hợp số âm bằng công thức 16 bit: $-2^{15} + (2^{15} - 1) - 18 = -19$. Kết quả đúng, giá trị được bảo toàn.

Nếu vô ý mở rộng số âm -19 theo kiểu thêm bit 0, ta được $0000\ 0000\ 1110\ 1101$ tức $+237$, một kết quả hoàn toàn sai. Đây là lỗi rất hay gặp khi lập trình ở mức thấp và khi chuyển kiểu dữ liệu giữa `signed char` và `int` trong ngôn ngữ C.

Hiện tượng tràn số

Khi kết quả của một phép toán vượt ra ngoài dải biểu diễn được với số bit hiện có thì xảy ra **tràn số** (*overflow*). Với số bù hai, quy tắc nhận biết rất gọn: *tràn số xảy ra khi cộng hai số cùng dấu mà kết quả lại mang dấu ngược lại*. Cộng hai số khác dấu thì không bao giờ tràn.

Ví dụ 1.17: Phát hiện tràn số

Thực hiện phép cộng hai số nguyên có dấu 8 bit $100 + 50$.

Giải. $100 = 0110\ 0100$ và $50 = 0011\ 0010$.

$$\begin{array}{r} 0110\ 0100\ (+100) \\ + 0011\ 0010\ (+50) \\ \hline 1001\ 0110\ (-106) \end{array}$$

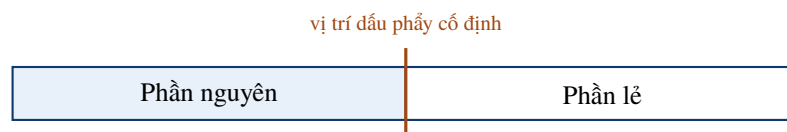
Hai toán hạng đều dương nhưng kết quả có bit dấu bằng 1, tức là số âm. Vậy đã xảy ra tràn số. Điều này hợp lý vì $150 > 127$, vượt quá dải biểu diễn của số có dấu 8 bit. Bộ xử lý sẽ bật cờ tràn OF trong thanh ghi trạng thái để chương trình biết mà xử lý.

Đáng chú ý là cũng dãy bit $1001\ 0110$ đó, nếu hiểu theo số không dấu thì bằng 150, hoàn toàn đúng. Tràn hay không tràn phụ thuộc vào việc chương trình đang hiểu dãy bit theo kiểu nào, chứ không phải vào bản thân phép cộng.

1.6.7 Biểu diễn số thực

Số dấu phẩy tĩnh

Ý tưởng ban đầu để biểu diễn số thực rất tự nhiên: dành một số bit cố định cho phần nguyên và số bit còn lại cho phần lẻ, giữa hai phần có một vị trí ngăn cách cố định. Cách này gọi là **số dấu phẩy tĩnh** (*fixed point*).



Hình 1.10. Khuôn dạng số dấu phẩy tĩnh

Vấn đề của cách này là không thể quyết định được tỷ lệ chia sao cho phù hợp với mọi bài toán. Dành nhiều bit cho phần nguyên thì mất độ chính xác của phần lẻ, dành nhiều bit cho phần lẻ thì không biểu diễn được số lớn. Thực tế về sau, dấu phẩy tĩnh chỉ còn được dùng với trường hợp đặc biệt là không có phần lẻ, tức là số nguyên, ứng với các kiểu khai báo `byte`, `word`, `integer`, `long integer`.

Số dấu phẩy động

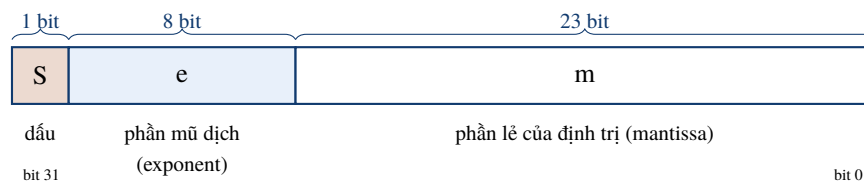
Số dấu phẩy động (*floating point number*) giải quyết vấn đề trên bằng cách để vị trí dấu phẩy tự thay đổi theo độ lớn của số. Tổng quát, một số thực X được biểu diễn dưới dạng:

$$X = \pm M \times R^E$$

trong đó M là phần định trị (*mantissa*), R là cơ số (*radix*) và E là phần mũ (*exponent*). Vì vị trí dấu phẩy trên phần định trị do phần mũ quyết định chứ không cố định, nên mới có tên gọi dấu phẩy động.

Chuẩn IEEE 754

Chuẩn IEEE 754, phiên bản năm 2008, quy định cách biểu diễn số dấu phẩy động được toàn bộ ngành công nghiệp máy tính tuân theo. Chuẩn dùng cơ số $R = 2$ và định nghĩa các dạng 32 bit, 64 bit và 128 bit. Ta xét kỹ dạng 32 bit, còn gọi là độ chính xác đơn (*single precision*).



Hình 1.11. Khuôn dạng số dấu phẩy động IEEE 754 dạng 32 bit

Ý nghĩa của ba trường:

- **S** là bit dấu: $S = 0$ ứng với số dương, $S = 1$ ứng với số âm.
- **e** (8 bit) là giá trị dịch chuyển của phần mũ E , liên hệ với nhau qua độ lệch (*bias*) bằng 127:

$$e = E + 127 \quad \Longleftrightarrow \quad E = e - 127.$$

- **m** (23 bit) là phần lẻ của phần định trị M , với $M = 1 + m$.

Từ đó công thức xác định giá trị của số thực là:

$$X = (-1)^S \times (1 + m) \times 2^{e-127}$$

Hai chi tiết trong công thức cần được giải thích, vì đây là chỗ sinh viên hay thắc mắc.

Thứ nhất, vì sao lại có số 1 trong biểu thức $1 + m$? Vì phần định trị luôn được chuẩn hóa về dạng $1,xxx$, tức là chữ số trước dấu phẩy luôn bằng 1. Đã luôn bằng 1 thì không cần lưu, nhờ vậy tiết kiệm được một bit mà vẫn không mất thông tin. Bit này gọi là bit ẩn (*hidden bit*). Thực chất ta có 24 bit độ chính xác trong khi chỉ tốn 23 bit lưu trữ.

Thứ hai, vì sao phải cộng độ lệch 127 vào phần mũ? Phần mũ E có thể âm hoặc dương. Nếu lưu E theo mã bù hai thì việc so sánh hai số dấu phẩy động sẽ phức tạp. Bằng cách cộng thêm 127, mọi giá trị e lưu trong trường đều không âm, và hai số dấu phẩy động cùng dấu có thể so sánh với nhau bằng đúng phép so sánh số nguyên không dấu trên toàn bộ 31 bit còn lại. Đây là một thủ thuật thiết kế rất khéo, giúp mạch so sánh đơn giản đi rất nhiều.

Ví dụ 1.18: Giải mã một số dấu phẩy động

Xác định giá trị của số thực được biểu diễn bằng 32 bit sau đây:

1100 0001 0101 0110 0000 0000 0000 0000

Giải. Tách dãy bit thành ba trường theo khuôn dạng hình 1.11:

Trường	Dãy bit	Giá trị
S (1 bit)	1	số âm
e (8 bit)	1000 0010	$128 + 2 = 130$
m (23 bit)	101 0110 0000 0000 0000 0000	0,671875

Tính phần mũ: $E = e - 127 = 130 - 127 = 3$.

Tính phần định trị: $M = 1 + m = 1,101011$ trong hệ nhị phân. Đổi sang thập phân,

$$m = 2^{-1} + 2^{-3} + 2^{-5} + 2^{-6} = 0,5 + 0,125 + 0,03125 + 0,015625 = 0,671875.$$

Thay vào công thức:

$$X = (-1)^1 \times (1 + 0,671875) \times 2^{130-127} = -1,671875 \times 8 = -13,375.$$

Có thể kiểm tra lại bằng cách dịch dấu phẩy trực tiếp trong hệ nhị phân:

$$X = -1,101011_2 \times 2^3 = -1101,011_2 = -(8 + 4 + 1 + 0,25 + 0,125) = -13,375.$$

Ví dụ 1.19: Mã hóa một số thực theo IEEE 754

Xác định dãy bit 32 bit biểu diễn số $-13,375$ theo chuẩn IEEE 754.

Giải. Thực hiện theo bốn bước.

Bước 1. Đổi sang nhị phân. Theo ví dụ 1.9 ta đã có $13,375 = (1101,011)_2$, nên $-13,375 = -1101,011_2$.

Bước 2. Chuẩn hóa. Dịch dấu phẩy về sau chữ số 1 đầu tiên:

$$-1101,011 = -1,101011 \times 2^3 \Rightarrow E = 3.$$

Bước 3. Xác định ba trường.

- Số âm nên $S = 1$.
- Giá trị dịch chuyển $e = E + 127 = 3 + 127 = 130 = 1000\ 0010$.
- Phần lẻ của định trị là các chữ số sau dấu phẩy của 1,101011, tức 101011, thêm các bit 0 vào bên phải cho đủ 23 bit: 101 0110 0000 0000 0000 0000.

Bước 4. Ghép lại.

1 | 1000 0010 | 101 0110 0000 0000 0000 0000

tức là

1100 0001 0101 0110 0000 0000 0000 0000 = (C1560000)₁₆.

Kết quả trùng khớp với ví dụ 1.18, đúng như mong đợi vì hai ví dụ là hai chiều của cùng một phép biến đổi.

Chú ý phân biệt hai chỗ hay nhầm: các bit 0 bù thêm vào phần định trị phải đặt ở *bên phải*, vì đó là các chữ số sau dấu phẩy có trọng số nhỏ dần; còn khi mở rộng bit cho số nguyên thì lại bù vào *bên trái*.

Các giá trị đặc biệt

Hai giá trị của trường e được chuẩn IEEE 754 dành riêng để mã hóa các trường hợp đặc biệt, không dùng cho số bình thường.

Bảng 1.11. Các trường hợp đặc biệt trong chuẩn IEEE 754 dạng 32 bit

Giá trị e	Phần định trị m	Ý nghĩa
$e = 0$	$m = 0$	Số không, có cả $+0$ và -0 tùy bit dấu
$e = 0$	$m \neq 0$	Số không chuẩn hóa (<i>denormalized</i>), giá trị $(-1)^S \times (0 + m) \times 2^{-126}$, dùng để biểu diễn các số rất nhỏ gần 0
$e = 255$	$m = 0$	Vô cùng, $+\infty$ hoặc $-\infty$, chẳng hạn kết quả của phép chia một số khác 0 cho 0
$e = 255$	$m \neq 0$	Không phải là số (NaN – <i>Not a Number</i>), chẳng hạn kết quả của $0/0$ hay $\sqrt{-1}$
$1 \leq e \leq 254$	tùy ý	Số chuẩn hóa bình thường, tính theo công thức đã nêu

Dải giá trị biểu diễn được

Xét trường hợp dùng 32 bit và so sánh hai chế độ.

Trong chế độ dấu phẩy tĩnh, nếu dùng cả 32 bit cho số nguyên dương thì số dương nhỏ nhất biểu diễn được là 1 và số lớn nhất xấp xỉ 2^{31} .

Trong chế độ dấu phẩy động theo IEEE 754, nếu tạm bỏ qua các mã dành riêng thì phần mũ chạy từ $0 - 127 = -127$ tới $255 - 127 = 128$, cho dải giá trị dương xấp xỉ từ 2^{-127} tới 2^{128} .

So sánh hai con số 2^{31} và 2^{128} cho thấy về khoảng biểu diễn, số dấu phẩy động vượt trội hơn hẳn số dấu phẩy tĩnh, dù cả hai đều dùng 32 bit. Lý do là số dấu phẩy động đã hy sinh độ phân giải để đổi lấy dải: các số biểu diễn được không nằm cách đều nhau như số dấu phẩy tĩnh mà thưa dần khi độ lớn tăng.

Các con số 2^{-127} và 2^{128} ở trên là kết quả tính thô khi cho e chạy hết dải 8 bit. Theo đúng chuẩn IEEE 754 thì $e = 0$ và $e = 255$ đã được dành riêng cho các trường hợp ở bảng 1.11, nên phần mũ thực sự của số chuẩn hóa chỉ chạy từ -126 tới $+127$. Do đó số dương chuẩn hóa nhỏ nhất là $2^{-126} \approx 1,18 \times 10^{-38}$ và số dương lớn nhất là $(2 - 2^{-23}) \times 2^{127} \approx 3,40 \times 10^{38}$. Khi làm bài tập, sinh viên cần nêu rõ mình đang dùng cách tính nào.

Bảng 1.12. So sánh hai dạng thông dụng của chuẩn IEEE 754

Đặc tính	32 bit (đơn)	64 bit (kép)
Số bit dấu	1	1
Số bit phần mũ	8	11
Số bit phần định trị	23	52
Độ lệch (bias)	127	1023
Dải phần mũ E	$-126 \dots +127$	$-1022 \dots +1023$
Giá trị dương lớn nhất	$\approx 3,4 \times 10^{38}$	$\approx 1,8 \times 10^{308}$
Số chữ số thập phân tin cậy	khoảng 7	khoảng 15
Kiểu tương ứng trong C	float	double

Làm tròn số thực và sai số

Khi biểu diễn một số trong chế độ dấu phẩy tĩnh, phần bị làm tròn là phần lẻ vượt quá số bit dành cho nó. Trong chế độ dấu phẩy động, phần bị mất là phần đuôi của định trị không đủ chỗ lưu.

Với số dấu phẩy động dùng n bit cho phần định trị, ta có hai loại sai số:

- **Sai số tuyệt đối** lớn nhất khi làm tròn số $x = \pm m_x 2^{E_x}$ tỷ lệ với 2^{E_x} , nghĩa là số càng lớn thì

sai số tuyệt đối càng lớn.

- **Sai số tương đối** luôn ở cỡ 2^{-n} , không phụ thuộc độ lớn của số. Với chuẩn IEEE 754 dạng 32 bit, $n = 23$ nên sai số tương đối cỡ $2^{-23} \approx 1,2 \times 10^{-7}$.

Việc sai số tương đối nhỏ và ổn định trên toàn dải chính là ưu điểm lớn nhất của số dấu phẩy động trong các bài toán tính gần đúng. Ngược lại, hệ quả cần nhớ là phép cộng dấu phẩy động không có tính kết hợp: $(a + b) + c$ và $a + (b + c)$ có thể cho kết quả khác nhau khi các số chênh lệch nhau nhiều về độ lớn.

1.6.8 Biểu diễn ký tự

Với k bit ta mã hóa được 2^k mã khác nhau, mỗi mã dùng cho một ký tự (*character*, tức là một ký hiệu bất kỳ, phân biệt với *letter* chỉ là một loại ký tự cụ thể là chữ cái). Lịch sử các bộ mã ký tự là quá trình mở rộng dần số bit để chứa được ngày càng nhiều hệ chữ viết.

Các bộ mã đầu tiên

Bộ mã EBCDIC (*Extended Binary Coded Decimal Interchange Code*) của IBM được dùng trên các máy lớn từ những năm 1960. Phiên bản tiền thân của nó dùng 6 bit, mã được 64 ký tự, chỉ đủ cho chữ in hoa, chữ số và một số dấu.

Bộ mã ASCII (*American Standard Code for Information Interchange*) dùng 7 bit, cho phép biểu diễn 128 ký tự. Cấu trúc của bảng mã được sắp xếp rất có chủ ý: 32 mã đầu tiên dành cho các mã điều khiển và truyền thông, tiếp theo là các dấu chính tả, các chữ số, các chữ in hoa, các chữ thường và các dấu đặc biệt.

Bảng 1.13. Bảng mã ASCII 7 bit

$\begin{matrix} b_6b_5b_4 \\ b_3b_2b_1b_0 \end{matrix}$	000	001	010	011	100	101	110	111
0000	NUL	DLE	SP	0	@	P	`	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	"	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	\$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	'	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	\	l	
1101	CR	GS	-	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	_	o	DEL

Ví dụ 1.20: Đọc bảng mã ASCII

Xác định mã ASCII của ký tự A và của ký tự a, rồi nhận xét.

Giải. Ký tự A nằm ở cột 100 và hàng 0001, nên mã của nó là $b_6b_5b_4b_3b_2b_1b_0 = 100\ 0001 = (41)_{16} = 65$.

Ký tự a nằm ở cột 110, hàng 0001, mã là $110\ 0001 = (61)_{16} = 97$.

Hai mã chênh nhau đúng $32 = 2^5$, tức chỉ khác nhau ở bit b_5 . Nhận xét này đúng cho mọi cặp chữ hoa – chữ thường trong bảng, và nó rất hữu ích khi lập trình: muốn đổi chữ hoa thành chữ thường chỉ cần bật bit b_5 , muốn đổi ngược lại chỉ cần xóa bit đó, không cần tra bảng.

Tương tự, mã của các chữ số 0 tới 9 là từ $(30)_{16}$ tới $(39)_{16}$, nên muốn lấy giá trị số từ mã ký tự chỉ cần trừ đi $(30)_{16}$. Những sắp xếp có quy luật như vậy trong bảng ASCII là kết quả của thiết kế có tính toán chứ không ngẫu nhiên.

Bộ **ASCII mở rộng** dùng trọn một byte cho mỗi ký tự nên biểu diễn được 256 ký tự. 128 mã tiếp theo được dùng cho chữ của các nước châu Âu, chữ Hy Lạp, chữ Slavơ. Tuy vậy khoảng trống này vẫn không đủ chỗ cho tiếng Việt, vốn cần thêm 141 ký tự riêng, và càng không thể đủ cho tiếng Trung Quốc với hơn 60 000 ký tự.

Bộ mã Unicode

Đối với các quốc gia có bộ chữ lớn như Trung Quốc, Nhật Bản, bộ mã 8 bit hoàn toàn không đủ chỗ. Nhật Bản đã khởi xướng một dự án lập bộ chữ dùng chung cho toàn cầu, gọi là UNICODE. Bộ chữ được chia trang cho các quốc gia; mặt chữ của một nước đã có sẵn sẽ được dùng lại trong các phần mềm khác. Về sau các tổ chức tiêu chuẩn đã chấp nhận UNICODE dưới chuẩn ISO 10646.

Mỗi quốc gia có thể nhận các trang mã (*code page*); mỗi ký tự được thể hiện qua mã của trang mã và số thứ tự của ký tự đó trong trang mã, gọi là điểm mã (*code point*), thường là một số hai byte.

Mã tiếng Việt

Trước khi có chuẩn thống nhất, đã từng tồn tại tới khoảng 40 bộ mã tiếng Việt 8 bit khác nhau, dẫn đến tình trạng loạn mã và không chia sẻ được dữ liệu giữa các hệ thống. Nguyên nhân gốc rễ đã nói ở trên: tiếng Việt cần 141 ký tự đặc thù trong khi vùng mã mở rộng của một byte chỉ còn 128 chỗ.

Bộ mã TCVN 5712/1993 là một giải pháp chấp vá gồm ba bộ mã khác nhau và vẫn không giải quyết được trọn vẹn:

- **Bộ mã 1** chiếm thêm một số chỗ trong vùng mã điều khiển, điều này nguy hiểm cho truyền thông vì các mã điều khiển có ý nghĩa riêng trong giao thức truyền.
- **Bộ mã 2** là bộ mã tổ hợp, dùng một chuỗi ký tự để thể hiện một chữ thuần Việt.
- **Bộ mã 3** hy sinh một số ký tự hoa có dấu, ví dụ chữ Æ.

Bộ mã tiêu chuẩn hiện hành là TCVN 6909/2001, thực chất là mã UNICODE, có hiệu lực từ ngày 1 tháng 1 năm 2003. Các cơ quan nhà nước bắt buộc phải dùng bộ mã này trong trao đổi dữ liệu. Chuẩn vẫn chấp nhận cả hai kiểu:

- **Mã dựng sẵn** (*precomposed*): coi chữ cái kèm dấu mũ và dấu thanh là một ký tự duy nhất, có một điểm mã riêng.
- **Mã tổ hợp** (*composite*): thể hiện một ký tự có dấu mũ, dấu thanh qua một chuỗi nhiều ký tự ghép lại.

Sự tồn tại song song của hai kiểu mã dựng sẵn và mã tổ hợp là nguyên nhân của một hiện tượng vẫn gặp cho tới nay: hai chuỗi tiếng Việt trông giống hệt nhau trên màn hình nhưng máy tính lại coi là khác nhau, khiến việc tìm kiếm hoặc so sánh cho kết quả sai. Cách xử lý là chuẩn hóa chuỗi về cùng một dạng trước khi so sánh, thao tác này trong các thư viện lập trình gọi là chuẩn hóa Unicode với các dạng NFC và NFD.

Phụ lục: Lịch sử phát triển của các bộ vi xử lý

Phần phụ lục này trình bày chi tiết hơn quá trình phát triển của bộ vi xử lý, bổ sung cho mục 1.4. Nội dung mang tính tham khảo, sinh viên nắm được các mốc chính và xu hướng chung là đủ.

Bốn thế hệ bộ vi xử lý

Thế hệ 1 (1971–1973)

Năm 1971, trong khi chế tạo các vi mạch dùng cho máy tính cầm tay, hãng Intel đã cho ra đời bộ vi xử lý đầu tiên là 4004 với 4 bit dữ liệu và 12 bit địa chỉ. Sau đó lần lượt xuất hiện 4040 (4 bit) và 8008 (8 bit) của Intel, PPS-4 (4 bit) của Rockwell International, IPM-16 (16 bit) của National Semiconductor. Đặc điểm chung của thế hệ này:

- Độ dài dữ liệu xử lý thường là 4 bit, cũng có loại dài hơn.
- Công nghệ chế tạo PMOS: mật độ tích hợp không cao, tốc độ thấp, giá thành rẻ và chỉ đưa ra được dòng tải nhỏ.
- Tốc độ thực hiện lệnh khoảng 10–60 micro giây một lệnh, tần số xung nhịp từ 0,1 tới 0,8 MHz.
- Tập lệnh đơn giản, cần nhiều vi mạch phụ trợ mới tạo nên một hệ vi xử lý hoàn chỉnh.

Thế hệ 2 (1974–1977)

Các bộ vi xử lý điển hình đều là loại 8 bit: 6502 của MOS Technology, 6800 và 6809 của Motorola, 8080 và 8085 của Intel, Z80 của Zilog. Tập lệnh phong phú hơn hẳn thế hệ trước. Khả năng định vị đạt 64 KB địa chỉ bộ nhớ, ngoài ra còn định vị được 256 địa chỉ vào–ra.

Các bộ vi xử lý này được ứng dụng rất nhiều trong công nghiệp và đặc biệt đã được dùng để chế tạo các máy vi tính 8 bit nổi tiếng như Apple II, Commodore, PC XT. Công nghệ chế tạo là NMOS hoặc CMOS, có mật độ tích hợp cao hơn PMOS và tiết kiệm điện năng, cho phép đạt tốc độ 1–8 micro giây một lệnh với tần số xung nhịp 1–5 MHz.

Thế hệ 3 (1978–1982)

Thế hệ này được đại diện bởi các bộ vi xử lý 16 bit: 8086, 80186, 80286 của Intel; 68000, 68010 của Motorola. Điểm tiến bộ hơn hẳn thế hệ 8 bit trước đó là tập lệnh đa dạng hơn với các lệnh nhân, chia và các lệnh xử lý chuỗi dữ liệu. Khả năng định vị địa chỉ bộ nhớ và vào–ra tăng lên rất nhiều, đạt 16 MB bộ nhớ và 64 KB không gian vào–ra đối với họ Intel.

Các bộ vi xử lý thế hệ này được dùng trong máy IBM PC, PC/AT và các máy Macintosh của Apple. Hầu hết được sản xuất bằng công nghệ HMOS, đạt tốc độ 0,1–1 micro giây một lệnh với tần số xung nhịp từ 5 tới 10 MHz.

Cũng trong thời gian này bắt đầu xuất hiện các bộ vi điều khiển 8 bit như 8048/8049 và 6805R2 (loại này có cả bộ biến đổi tương tự – số 12 bit), hoặc các bộ vi điều khiển 4 bit như MC141000 và 1 bit như MC14500B.

Thế hệ 4 (từ 1983)

Đại diện cho thế hệ này là các bộ vi xử lý 32 bit 80386, 80486 và 64 bit Pentium của Intel, các bộ vi xử lý 32 bit 68020, 68030, 68040, 68060 của Motorola. Các bộ vi xử lý thế hệ này đều có 32 bit địa chỉ, định vị được 4 GB bộ nhớ vật lý, ngoài ra còn có chế độ làm việc với bộ nhớ ảo.

Nhiều cấu trúc và cơ chế hoạt động vốn chỉ có trong máy tính lớn đã được áp dụng cho bộ vi xử lý, chẳng hạn cơ chế đường ống (*pipeline*) và bộ nhớ cache. Các bộ vi xử lý này đều tích hợp sẵn khối quản lý bộ nhớ MMU và bộ đồng xử lý toán học bên trong. Nhờ những cải tiến đó, máy vi tính dùng vi xử lý thế hệ này đã cạnh tranh được với máy tính mini trong rất nhiều ứng dụng. Công nghệ chế tạo chủ yếu là HCMOS.

Bên cạnh dòng vi xử lý xây dựng theo cấu trúc truyền thống với tập lệnh đầy đủ (CISC – *Complex Instruction Set Computer*) nói trên, thời gian này cũng xuất hiện các bộ vi xử lý với tập lệnh rút gọn (RISC – *Reduced Instruction Set Computer*), có khả năng thực hiện lệnh với tốc độ cao hơn và nhiều tính năng sánh được với máy tính lớn thế hệ trước. Tiêu biểu là Alpha của Digital, PowerPC của liên minh Apple – Motorola – IBM.

Dòng vi xử lý Intel

Khởi đầu từ tháng 10 năm 1971

Intel bắt đầu phát triển bộ vi xử lý đầu tiên vào năm 1969. Đây là một phần của dự án thiết kế một họ vi mạch tính toán lập trình được cho hãng Busicom tại Nhật Bản. Ban đầu Busicom đề nghị trả cho Intel 60 000 đô-la để thiết kế một vi mạch đóng vai trò bộ não, nhưng Intel đề nghị dùng nguồn vốn này để thiết kế ra một bộ vi xử lý.

Busicom đồng ý, và Intel giới thiệu ra thị trường bộ vi xử lý 4004 vào ngày 15 tháng 10 năm 1971, bán được 200 bộ. Chìa khóa thành công của bộ vi xử lý nằm ở ý tưởng cung cấp một thiết bị có thể lập trình bằng phần mềm; trước đó các vi mạch số đều được thiết kế với chức năng cố định.

8086 và 8088 (cuối những năm 1970)

- Địa chỉ hóa được 1 MB bộ nhớ trong khi các bộ xử lý cùng thời chỉ quản lý được 64 KB.
- Bus dữ liệu 16 bit, ưu điểm rõ rệt so với các bộ vi xử lý đương thời.
- 8088 là bản thay thế của 8086 với bus dữ liệu rút xuống 8 bit nhằm giảm giá thành hệ thống.
- 8088 là bộ vi xử lý đầu tiên được dùng cho máy tính IBM.

80186

Lõi xử lý vẫn là 8086 nhưng vi mạch được tích hợp thêm cả các mạch phụ trợ. Loại này không được dùng cho máy tính cá nhân mà được thương mại hóa cho các hệ thống điều khiển máy móc thiết bị trong công nghiệp.

80286 (năm 1983)

Được thiết kế tương thích với 8088 để chạy được các chương trình viết cho máy PC-XT vốn đã phổ biến trên thị trường. Vì vậy nó có hai chế độ hoạt động:

- **Chế độ thực**, tương thích 8088.
- **Chế độ bảo vệ** (*protected mode*), cung cấp các tính năng mạnh: truy cập tới 16 MB bộ nhớ, nhưng cần một hệ điều hành đặc biệt để khai thác. Trên thực tế hầu hết người dùng khi đó chỉ có hệ điều hành MS-DOS nên chế độ này ít được tận dụng.

80286 được thương mại hóa cùng dòng máy PC-AT.

386DX (năm 1985)

Đây là bộ vi xử lý 32 bit đầu tiên của Intel, tất cả các bus đều 32 bit. Nó chạy được ở chế độ thực và chế độ bảo vệ của 286, đồng thời có chế độ bảo vệ riêng mạnh hơn với hai chức năng mới:

- Quản lý bộ nhớ ảo (*virtual memory*): dùng ổ đĩa cứng làm phần mở rộng, cho phép máy tính quản lý tới 4 GB không gian nhớ.
- Chế độ ảo 8086, phục vụ việc chạy hệ điều hành DOS.

386SX (năm 1988)

Khác biệt so với 386DX:

- Bus dữ liệu giảm xuống 16 bit.
- Bus địa chỉ giảm xuống 24 bit, giới hạn truy cập bộ nhớ còn 16 MB.
- Bản dùng cho máy tính xách tay thông dụng đầu tiên có tên 386SL, chạy với nguồn 3,3 V.
- Mục tiêu chính là giảm giá thành máy tính.

486DX (năm 1989) và 486SX (năm 1991)

Khác biệt của 486DX so với họ 386:

- Tích hợp sẵn mạch đồng xử lý toán học (*coprocessor*), cung cấp khả năng tính toán dấu phẩy động với hiệu suất cao.
- Có 8 KB bộ nhớ đệm (*cache*). Đây là bộ nhớ SRAM chứa mã lệnh đã đọc trước đó, giúp CPU tránh phải lấy lệnh từ DRAM vốn có tốc độ truy xuất chậm hơn.

486SX giống 486DX nhưng không tích hợp bộ đồng xử lý toán học, nhằm hạ giá thành.

Pentium (năm 1993)

- Bus dữ liệu ngoài 64 bit, bên trong CPU chia thành hai đường ống 32 bit.
- Có bộ nhớ đệm 8 KB ghi được dành cho chương trình.
- Hầu hết các bộ xử lý Pentium chạy với nguồn 3,3 V để tăng tốc độ hoạt động.
- Bao gồm bộ nhân xung nhịp gấp đôi, chọn bằng cầu nối.
- Phần lớn dùng kiểu chân SPGA (*Standard Pin Grid Array*), cho phép tăng mật độ tích hợp trong chip.

Pentium sau năm 1996

- Tập lệnh MMX hỗ trợ xử lý đa phương tiện.
- Tăng hệ số nhân xung nhịp, lên tới 45 lần.
- Cải thiện tốc độ xử lý bằng cách dự đoán hướng rẽ nhánh và lưu kết quả dự đoán trong cache.
- Cải tiến cấu trúc siêu phân luồng.
- Bổ sung các tập lệnh SSE và SSE2.

Dòng Intel Core

Từ năm 2006, Intel chuyển sang kiến trúc Core với nhiều lõi xử lý trên cùng một chip. Dòng Core i được đánh số theo thế hệ, mỗi thế hệ cải tiến về vi kiến trúc, công nghệ chế tạo, dung lượng cache và số lõi; tới năm 2022 đã có bộ vi xử lý Intel Core i thế hệ thứ 12.

Bảng 1.14. Tổng hợp một số mốc của dòng vi xử lý Intel

Bộ vi xử lý	Năm	Độ dài dữ liệu	Điểm nổi bật
4004	1971	4 bit	Bộ vi xử lý đầu tiên trên thế giới
8080	1974	8 bit	Tập lệnh phong phú, quản lý 64 KB
8086	1978	16 bit	Quản lý 1 MB, khởi đầu kiến trúc x86
8088	1979	16 bit	Bus dữ liệu ngoài 8 bit, dùng cho IBM PC
80286	1983	16 bit	Chế độ bảo vệ, quản lý 16 MB
386DX	1985	32 bit	Bộ vi xử lý 32 bit đầu tiên, bộ nhớ ảo
486DX	1989	32 bit	Tích hợp đồng xử lý toán học và cache 8 KB
Pentium	1993	32 bit	Hai đường ống, bus dữ liệu ngoài 64 bit
Core	2006	64 bit	Nhiều lõi trên một chip

Câu hỏi ôn tập

1. Nêu bốn chức năng cơ bản của máy tính. Chức năng nào làm nên bản chất "vạn năng" của máy tính và vì sao?
2. Phân biệt dữ liệu và thông tin. Cho một ví dụ trong đó cùng một dãy bit được hiểu theo ba cách khác nhau.
3. So sánh máy tính cá nhân, máy chủ và máy tính nhúng theo mục đích sử dụng, tiêu chí thiết kế và sản lượng.
4. Trình bày vai trò của CPU, bộ nhớ chính và hệ thống vào–ra trong mô hình cơ bản của máy tính.
5. Phân biệt ROM và RAM. Vì sao tên gọi RAM dễ gây hiểu nhầm?
6. Kể tên ba loại bus hệ thống và cho biết chiều truyền tín hiệu của từng loại.
7. Số đường của bus địa chỉ ảnh hưởng đến thông số nào của hệ thống? Còn số đường của bus dữ liệu thì ảnh hưởng đến thông số nào?
8. Phân biệt ba mức nghiên cứu: kiến trúc tập lệnh, tổ chức máy tính và phần cứng. Vì sao nói kiến trúc tập lệnh là một bản giao ước?
9. Mô tả chu trình lệnh. Vai trò của thanh ghi PC và thanh ghi IR là gì?
10. Vì sao nội dung PC được tăng ngay trong pha nhận lệnh chứ không phải sau khi thực hiện xong lệnh?
11. Nêu bốn nhóm thao tác cơ bản mà một lệnh có thể yêu cầu bộ xử lý thực hiện.
12. Ngắt là gì? Vì sao cơ chế ngắt lại cần thiết đối với máy tính đa nhiệm?

13. Trình bày các bước bộ xử lý thực hiện khi nhận được tín hiệu ngắt.
14. So sánh xử lý ngắt tuần tự và xử lý ngắt theo mức ưu tiên. Ngắt lồng nhau xảy ra trong trường hợp nào?
15. So sánh ba phương pháp vào-ra: bằng chương trình, bằng ngắt và DMA. Trường hợp nào bắt buộc phải dùng DMA?
16. Nêu bốn thế hệ máy tính cùng công nghệ linh kiện đặc trưng của từng thế hệ.
17. Trình bày các thông số kỹ thuật chính của máy ENIAC. Hạn chế lớn nhất của ENIAC là gì?
18. Phát biểu ba nguyên lý của chương trình lưu trữ. Hệ quả quan trọng nhất của các nguyên lý này đối với phần mềm là gì?
19. Mô tả tổ chức bộ nhớ và khuôn dạng lệnh của máy IAS. Thanh ghi IBR có tác dụng gì?
20. Điểm nghẽn von Neumann là gì? Nêu ba kỹ thuật dùng để làm nhẹ điểm nghẽn này.
21. So sánh kiến trúc von Neumann và kiến trúc Harvard. Vì sao bộ xử lý hiện đại lại dùng kiến trúc Harvard cải tiến?
22. Vì sao máy tính dùng hệ nhị phân mà không dùng hệ thập phân? Hệ thập lục phân được dùng để làm gì?
23. Trình bày nguyên tắc biểu diễn số nguyên có dấu theo mã bù hai. Vì sao dải biểu diễn của mã bù hai không đối xứng?
24. Phân biệt mở rộng theo số không dấu và mở rộng theo số có dấu. Điều gì xảy ra nếu dùng nhầm?
25. Nêu quy tắc nhận biết tràn số khi cộng hai số bù hai.
26. Giải thích ý nghĩa của bit ẩn và của độ lệch 127 trong chuẩn IEEE 754 dạng 32 bit.
27. Vì sao số dấu phẩy động có dải biểu diễn rộng hơn nhiều so với số dấu phẩy tính cùng độ dài? Cái giá phải trả là gì?
28. Trình bày sự phát triển của các bộ mã ký tự từ ASCII tới Unicode. Vì sao một byte không đủ cho tiếng Việt?
29. Phân biệt mã dựng sẵn và mã tổ hợp trong biểu diễn tiếng Việt. Vấn đề thực tế nào nảy sinh từ việc tồn tại song song hai kiểu mã?

Bài tập

A. Chuyển đổi hệ đếm

1. Đổi các số thập phân sau sang hệ nhị phân và hệ thập lục phân: a) 200 b) 1000 c) 45,75
2. Đổi các số nhị phân sau sang hệ thập phân: a) 1010 1100 b) 1111 0001 c) 1011, 101
3. Đổi các số thập lục phân sau sang hệ thập phân và hệ nhị phân: a) $3F_{16}$ b) $A5_{16}$ c) $2FE_{16}$
4. Một bộ vi xử lý có bus địa chỉ 24 đường. Xác định dung lượng bộ nhớ tối đa mà nó quản lý được, biết mỗi ngăn nhớ chứa một byte.

B. Biểu diễn số nguyên

5. Biểu diễn các số sau bằng 8 bit theo dạng số nguyên không dấu: a) 100 b) 255 c) 78
6. Biểu diễn các số sau bằng 8 bit theo mã bù hai: a) +100 b) -100 c) -1 d) -128
7. Xác định giá trị của các dãy bit 8 bit sau trong hai trường hợp: hiểu là số không dấu và hiểu là số bù hai. a) 1000 0000 b) 0111 1111 c) 1111 0000
8. Tìm số bù một và số bù hai của 0011 0101. Kiểm tra kết quả bằng công thức $2^n - A$.
9. Số -45 được biểu diễn bằng 8 bit theo mã bù hai. Hãy mở rộng nó thành dạng 16 bit và giải thích quy tắc đã dùng.
10. Thực hiện các phép cộng sau trên số bù hai 8 bit và cho biết trường hợp nào xảy ra tràn số: a) $70 + 80$ b) $(-70) + (-80)$ c) $100 + (-30)$

C. Biểu diễn số thực

11. Biểu diễn các số thực sau theo chuẩn IEEE 754 dạng 32 bit, viết kết quả dưới dạng nhị phân và dạng thập lục phân: a) 25,5 b) -0,75
12. Xác định giá trị thập phân của số thực được biểu diễn bằng 32 bit sau:

0100 0001 1100 1000 0000 0000 0000 0000

13. Xác định số dương chuẩn hóa nhỏ nhất và số dương lớn nhất biểu diễn được theo chuẩn IEEE 754 dạng 32 bit.
14. Một số thực được lưu theo chuẩn IEEE 754 dạng 32 bit. Ước lượng sai số tương đối khi làm tròn và cho biết số chữ số thập phân đáng tin cậy.

D. Biểu diễn ký tự và tổng hợp

15. Tra bảng 1.13 để xác định mã ASCII của chuỗi ký tự PTIT, viết kết quả dưới dạng nhị phân và thập lục phân.
16. Dãy byte sau là mã ASCII của một chuỗi ký tự, viết theo hệ mười sáu. Hãy giải mã chuỗi đó:

$$48 \quad 65 \quad 6C \quad 6C \quad 6F$$
17. Xét máy tính giả định ở ví dụ 1.3. Cho biết nội dung bộ nhớ ban đầu là $M(400) = 1500$, $M(401) = 5501$, $M(402) = 2502$, $M(500) = 000A$, $M(501) = 0006$, và $PC = 400$. Hãy lập bảng theo dõi diễn biến của chương trình và cho biết giá trị cuối cùng của $M(502)$.
18. Một hệ thống có CPU thực hiện 2×10^9 lệnh mỗi giây, cần đọc một khối dữ liệu 4 MB từ ổ đĩa có tốc độ truyền 200 MB/s. Giả sử mỗi lần xử lý ngắt tốn 400 lệnh và mỗi lần ngắt chuyển được 4 KB. Hãy tính số lần ngắt, tổng số lệnh dành cho xử lý ngắt và tỷ lệ phần trăm năng lực CPU bị chiếm trong khoảng thời gian truyền. Nhận xét về việc nên dùng DMA hay không.

Đáp án và hướng dẫn giải

A. Chuyển đổi hệ đếm

Bài 1. a) $200 = 128 + 64 + 8 = 11001000_2 = (C8)_{16}$.

b) $1000 = 512 + 256 + 128 + 64 + 32 + 8 = 1111101000_2 = (3E8)_{16}$.

c) Phần nguyên $45 = 32 + 8 + 4 + 1 = 101101_2$; phần lẻ $0,75 = 0,11$. Vậy $45,75 = (101101,11)_2$. Nhóm bốn bit từ dấu phẩy ra hai phía: $00101101.1100 \Rightarrow (2D,C)_{16}$.

Bài 2. a) $128 + 32 + 8 + 4 = 172$. b) $128 + 64 + 32 + 16 + 1 = 241$. c) $8 + 2 + 1 + 0,5 + 0,125 = 11,625$.

Bài 3. a) $(3F)_{16} = 3 \times 16 + 15 = 63 = 00111111_2$.

b) $(A5)_{16} = 10 \times 16 + 5 = 165 = 10100101_2$.

c) $(2FE)_{16} = 2 \times 256 + 15 \times 16 + 14 = 766 = 001011111110_2$.

Bài 4. $2^{24} = 16777216 \text{ byte} = 16 \text{ MB}$.

B. Biểu diễn số nguyên

Bài 5. a) $100 = 01100100_2$. b) $255 = 11111111_2$. c) $78 = 01001110_2$.

Bài 6. a) $+100 = 01100100_2$.

b) -100 : lấy bù hai của 01100100_2 . Giữ nguyên đuôi 100, đảo phần còn lại 01100 thành 10011 , được 10011100_2 . Kiểm tra: $-128 + 16 + 8 + 4 = -100$.

c) $-1 = 1111\ 1111$. d) $-128 = 1000\ 0000$.

Bài 7.

Dãy bit	Hiểu là không dấu	Hiểu là bù hai
1000 0000	128	-128
0111 1111	127	+127
1111 0000	240	-16

Bài này cho thấy rõ: bản thân dãy bit không mang dấu, dấu là do cách chương trình diễn giải.

Bài 8. $A = 0011\ 0101 = 53$. Bù một là $1100\ 1010$, bù hai là $1100\ 1011$. Kiểm tra: $2^8 - 53 = 203$, và $1100\ 1011 = 128 + 64 + 8 + 2 + 1 = 203$. Đúng.

Bài 9. $45 = 0010\ 1101$, lấy bù hai được $-45 = 1101\ 0011$. Vì là số có dấu và số này âm nên mở rộng bằng cách thêm các bit 1 vào bên trái: $-45 = 1111\ 1111\ 1101\ 0011$ (16 bit).

Bài 10.

Phép cộng	Kết quả 8 bit	Nhận xét
a) $0110\ 0100 + 0101\ 0000$	$1001\ 0110 = -106$	Hai số dương, kết quả âm: tràn
b) $1011\ 1010 + 1011\ 0000$	$0110\ 1010 = +106$	Hai số âm, kết quả dương: tràn
c) $0110\ 0100 + 1110\ 0010$	$0100\ 0110 = +70$	Khác dấu: không bao giờ tràn

Ở câu b) và c) có bit nhớ đi ra khỏi bit cao nhất, nhưng bit nhớ này bị bỏ qua và không phải là dấu hiệu của tràn số đối với số có dấu.

C. Biểu diễn số thực

Bài 11.

a) $25,5 = (11001,1)_2 = 1,10011 \times 2^4$. Do đó $S = 0$, $e = 4 + 127 = 131 = 1000\ 0011$, $m = 100\ 1100\dots 0$. Ghép lại:

$$0100\ 0001\ 1100\ 1100\ 0000\ 0000\ 0000\ 0000 = (41CC0000)_{16}.$$

b) $-0,75 = -(0,11)_2 = -1,1 \times 2^{-1}$. Do đó $S = 1$, $e = -1 + 127 = 126 = 0111\ 1110$, $m = 100\dots 0$. Ghép lại:

$$1011\ 1111\ 0100\ 0000\ 0000\ 0000\ 0000\ 0000 = (BF400000)_{16}.$$

Bài 12. Tách trường: $S = 0$; $e = 1000\ 0011 = 131$ nên $E = 4$; $m = 1001\ 0\dots = 2^{-1} + 2^{-4} = 0,5625$. Vậy

$$X = (+1) \times (1 + 0,5625) \times 2^4 = 1,5625 \times 16 = 25,0.$$

Bài 13. Số dương chuẩn hóa nhỏ nhất ứng với $e = 1, m = 0$: $2^{-126} \approx 1,18 \times 10^{-38}$. Số dương lớn nhất ứng với $e = 254, m = 1 - 2^{-23}$: $(2 - 2^{-23}) \times 2^{127} \approx 3,40 \times 10^{38}$.

Bài 14. Phần định trị có 23 bit lưu trữ cộng bit ẩn, sai số tương đối cỡ $2^{-24} \dots 2^{-23} \approx 6 \times 10^{-8} \dots 1,2 \times 10^{-7}$. Vì 10^{-7} ứng với khoảng 7 chữ số thập phân nên dạng 32 bit cho khoảng 7 chữ số đáng tin cậy. Đây là lý do khi cần độ chính xác cao hơn phải dùng dạng 64 bit.

D. Biểu diễn ký tự và tổng hợp

Bài 15.

Ký tự	P	T	I
Nhị phân	101 0000	101 0100	100 1001
Thập lục phân	50 ₁₆	54 ₁₆	49 ₁₆

Chuỗi PTIT có mã 50₁₆ 54₁₆ 49₁₆ 54₁₆.

Bài 16. 48₁₆ = H, 65₁₆ = e, 6C₁₆ = l, 6C₁₆ = l, 6F₁₆ = o. Chuỗi cần tìm là Hello.

Bài 17. Chương trình cộng nội dung hai ngăn 500 và 501 rồi cất kết quả vào 502. Vì $M(500) = (000A)_{16} = 10$ và $M(501) = 6$:

Bước	Pha	PC	IR	AC	Diễn giải
1	Nhận lệnh	400 → 401	1500	–	Đọc lệnh tại 400
2	Thực hiện	401	1500	000A	$AC \leftarrow M(500) = 10$
3	Nhận lệnh	401 → 402	5501	000A	Đọc lệnh tại 401
4	Thực hiện	402	5501	0010	$AC \leftarrow 10 + 6 = 16$
5	Nhận lệnh	402 → 403	2502	0010	Đọc lệnh tại 402
6	Thực hiện	403	2502	0010	$M(502) \leftarrow 16$

Kết quả: $M(502) = (0010)_{16} = 16$ trong hệ thập phân.

Bài 18. Số lần ngắt: $4 \text{ MB} / 4 \text{ KB} = 1024$ lần. Tổng số lệnh dành cho xử lý ngắt: $1024 \times 400 = 409\,600$ lệnh.

Thời gian truyền khối: $4 \text{ MB} / 200 \text{ MB/s} = 0,02$ giây. Trong khoảng đó CPU thực hiện được $2 \times 10^9 \times 0,02 = 4 \times 10^7$ lệnh. Tỷ lệ chiếm dụng:

$$\frac{409\,600}{4 \times 10^7} \approx 1,02\%.$$

Nhận xét: mức chiếm dụng khoảng 1% nghe có vẻ nhỏ, nhưng cần nhớ đây chỉ là một thao tác đọc đĩa đơn lẻ. Với máy chủ phục vụ hàng nghìn thao tác vào–ra mỗi giây, chi phí này cộng dồn lại sẽ chiếm phần đáng kể năng lực CPU. Dùng DMA sẽ giảm số lần ngắt xuống còn một

lần cho cả khối, đưa chi phí về mức không đáng kể. Vì vậy mọi bộ điều khiển ổ đĩa và card mạng hiện nay đều dùng DMA.

HẾT CHƯƠNG 1